

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE



**Academic Strengthening & Advancement
(AS&A)**



Department of Computer Science and Engineering

R – 23

III B.Tech. II Semester

Learning Material

On

Fundamentals of Java Programming

Prepared by:

Faculty of CSE

FUNDAMENTALS OF JAVA PROGRAMMING**III Year – II Semester**

Lecture : 3

Internal Marks : 30

Credits : 3

External Marks : 70

Course Objectives

- To identify Java language components and how they work together in applications.
- To learn the fundamentals of object-oriented programming in Java, including defining classes, invoking methods, using class libraries.
- To learn how to extend Java classes with inheritance and dynamic binding and how to use exception handling in Java applications.
- To understand how to design applications with threads in Java.
- To understand how to use Java APIs for program development.

Course Outcomes

Upon successful completion of the course, the students will be able to

- design, write, and debug Java programs using various data types, control statements, and operators.
- apply object-oriented concepts to build standard java application.
- utilize arrays, implement various inheritance techniques, and work with interfaces in Java to create object-oriented programs.
- build applications using packages, exception handling, Java I/O.
- develop dynamic applications with strings, multi-threading, JDBC and JFX.

Course Content**UNIT – I: Object Oriented Programming**

Basic concepts, Principles, Program Structure in Java: Introduction, Writing Simple Java Programs, Elements or Tokens in Java Programs, Java Statements, Command Line Arguments. Data Types, **Variables, and Operators:** Introduction, **Data Types** in Java, Declaration of Variables, Data Types, Constants. **Introduction to Operators**, Precedence and Associativity of Operators, Assignment Operator (=), Basic Arithmetic Operators, Increment (++) and Decrement (- -) Operators, Ternary Operator, Relational Operators, Boolean Logical Operators, Bitwise Logical Operators. **Control Statements:** Introduction, if Expression, Nested if Expressions, if-else Expressions, Ternary Operator?:, Switch Statement, Iteration Statements, while Expression, do-while Loop, for Loop, Nested for Loop, For-Each for Loop, Break Statement, Continue Statement.

UNIT – II: Classes and Objects

Introduction, Class Declaration and Modifiers, Class Members, Declaration of Class Objects, Constructor Methods for Class, Keyword this. Methods: Introduction, Defining Methods, Overloaded Methods, Attributes Final and Static.

UNIT – III: Arrays

Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Accessing Elements of Arrays.

Inheritance: Introduction, Process of Inheritance, Types of Inheritances, Multilevel Inheritance, Application of Keyword Super, Method Overriding, Dynamic Method Dispatch, Abstract Classes, Final class and Methods.

Interfaces: Introduction, Declaration of Interface, Implementation of Interface, Multiple Interfaces, Nested Interfaces.

UNIT – IV: Packages and Java Library

Introduction, Defining Package, Importing Packages and Classes into Programs, Path and Class Path, Access Control.

Exception Handling: Introduction, Hierarchy of Standard Exception Classes, Keywords throws and throw, try, catch, and finally Blocks, Multiple Catch Clauses, Class Throwable, Unchecked Exceptions, Checked Exceptions.

UNIT – V: Multithreading and Event Handling

Multithreaded Programming: Introduction, Need for Multiple Threads Multithreaded Programming for Multi-core Processor, Main Thread-Creation of New Threads, Thread Priority-Synchronization, Inter-thread Communication - Suspending, Resuming, and Stopping of Threads.

Event Handling- Events, event sources, event classes, event listeners, delegation event model, handling mouse and keyboard events.

Text Books:

1. Debasis Samanta, Monalisa Sarma, Joy with JAVA: Fundamentals of Object-Oriented Programming, Cambridge, 2023.
2. Paul Deitel, Harvey Deitel, JAVA9 for Programmers, 4th Edition, Pearson.

References Books:

1. Herbert Schildt, The complete Reference Java, 11th Edition, TMH.
2. Y Daniel Liang, Introduction to Java programming, 7th Edition, Pearson.

e-Learning Resources:

1. <https://nptel.ac.in/courses/106/105/106105191/>
2. https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_012880464547618816347_shared/overview

UNIT- 1**Object Oriented Programming through Java****Objective:**

1. To get acquainted with the concepts of object-oriented programming.

Syllabus:

Module-1: Basic concepts, Principles, Program Structure in Java: Introduction, Writing Simple Java Programs, Elements or Tokens in Java Programs, Java Statements, Command Line Arguments.

Module 2: Data Types, Variables, and Operators: Introduction, Data Types in Java, Declaration of Variables, Data Types, Type Casting, Scope of Variable Identifier, Literal Constants, Symbolic Constants, Introduction to Operators, Precedence and Associativity of Operators, Assignment Operator (=), Basic Arithmetic Operators, Increment (++) and Decrement (- -) Operators, Ternary Operator, Relational Operators, Boolean Logical Operators, Bitwise Logical Operators.

Module 3: Control Statements: Introduction, if Expression, Nested if Expressions, if-else Expressions, Ternary Operator? Switch Statement, Iteration Statements, while Expression, do-while Loop, for Loop, Nested for Loop, for-Each for Loop, Break Statement, Continue Statement.

Learning Outcomes:

Students will be able to

- . Understand the principles of object-oriented programming.
- . Understand Syntax of basic Java Programming Constructs and apply them in writing simple programs.
- . Distinguish between Implicit and Explicit Casting.

Module-1

1.Object Oriented Programming

Basic concepts:

- . OOP treats data as a critical element in the program development and does not allow it to flow freely around the system. It ties data more closely to the function that operate on it, and protects it from accidental modification from outside function.
- . OOP allows decomposition of a problem into a number of entities called objects and then builds data and function around these objects.
- . The organization of data and function in object-oriented programs is shown in the below figure

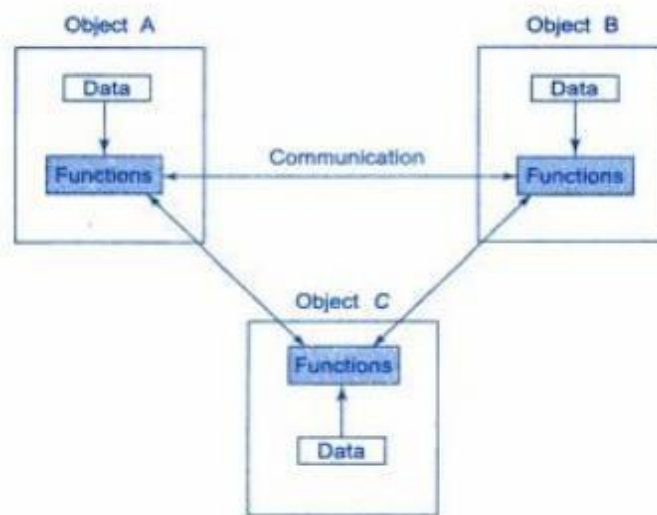


Fig:1.1 Organization of Data and Functions in OOP

- . The data of an object can be accessed only by the function associated with that object. However, function of one object can access the function of other objects.

1.1 Principles of OOP:

The following are the general concepts of OOP

1. Object.
2. Class.
3. Data abstraction.
4. Data Encapsulation
5. Inheritance
6. Polymorphism

7. Dynamic binding
8. Message passing.

1. **OBJECT:**

- Objects are the basic run time entities in an object-oriented system.
- They may represent a person, a place, a bank account, a table of data or any item that the program has to handle.
- Objects take up space in the memory and have an associated address.
- When a program is executed, the objects interact by sending messages to one another.
- For example, if “customer” and “account” are to object in a program, then the customer object may send a message to the count object requesting for the bank balance.
- Each object contain data, and code to manipulate data.

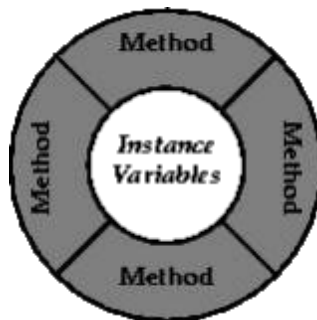


Fig:1.2 Object = Data+Methods

2. **CLASS:**

- The entire set of data and code of an object can be made a user-defined data type with the help of class.
- Objects are variables of the type class.
- Once a class has been defined, we can create any number of objects belonging to that class. Each object is associated with the data of type class with which they are created.
- ***A class is thus a collection of objects similar types. Object is an Instance of a Class***
- Classes are user-defined data types and behave like the built-in types of a programming language.

3. **DATA ABSTRACTION:**

- Abstraction refers to the act of representing essential features without including the background details or explanation.
- Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight, and cost, and functions operate on these attributes.

- They encapsulate all the essential properties of the object that are to be created.
- The attributes are sometimes called data members because they hold information.
- The functions that operate on these data are sometimes called methods or member function.

4. **DATA ENCAPSULATION:**

- The wrapping up of data and functions into a single unit (called class) is known as encapsulation.
- Data and encapsulation is the most striking feature of a class. The data is not accessible to the outside world, and only those functions which are wrapped in the class can access it.
- These functions provide the interface between the object's data and the program.
- This insulation of the data from direct access by the program is called data hiding or information hiding.

5. **INHERITANCE:**

- Inheritance is the process by which objects of one class acquired the properties of objects of another classes.
- In OOP, the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. This is possible by deriving a new class from the existing one. The new class will have the combined feature of both the classes.

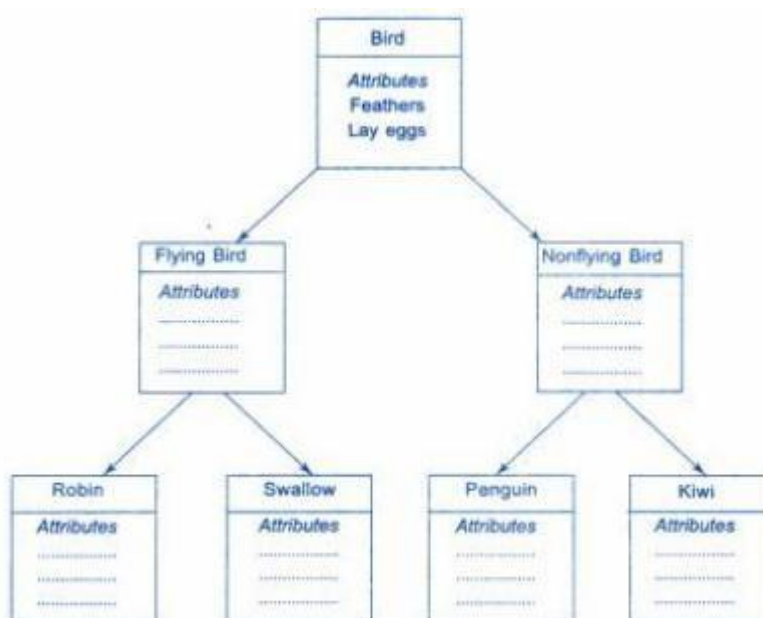


Fig:1.1.1 Inheritance

6. **POLYMORPHISM:**

- Polymorphism, a Greek term, means the ability to take more than one form.
- An operation may exhibit different behavior in different instances. The behavior depends upon the types of data used in the operation.

- The below figure illustrates that a single function name can be used to handle different number and different types of argument.

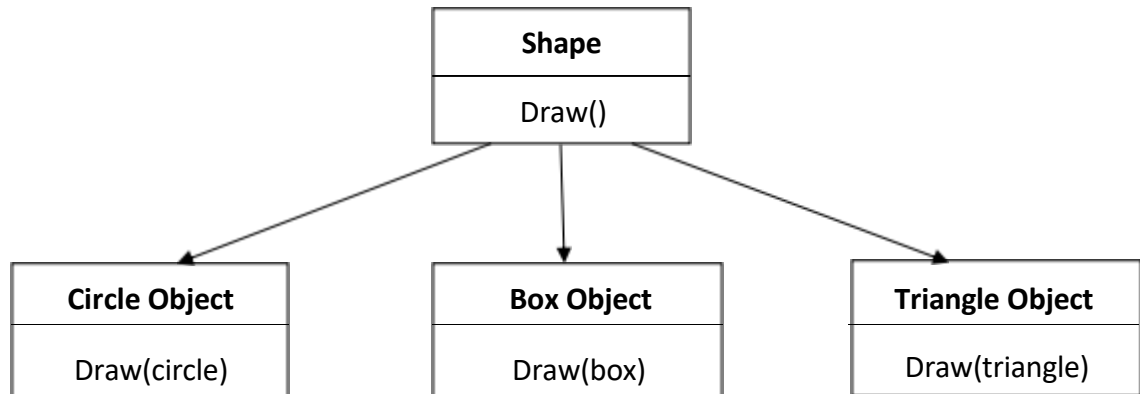


Fig:1.1.2 Polymorphism

7. DYNAMIC BINDING:

- Binding refers to the linking of a procedure call to the code to be executed in response to the call.
- Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time.
- It is associated with polymorphism and inheritance.
- A function call associated with a polymorphic reference depends on the dynamic type of that reference.

8. MESSAGE PASSING:

- An object-oriented program consists of a set of objects that communicate with each other.
- The process of programming in an object-oriented language, involves the following basic steps:
 1. Creating classes that define object and their behavior
 2. Creating objects from class definitions, and
 3. Establishing communication among objects.
- Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another.
- A Message for an object is a request for execution of a procedure, and therefore will invoke a function (procedure) in the receiving object that generates the desired results.
- Message passing involves specifying the name of object, the name of the function (message) and the information to be sent.

1.2 JAVA VIRTUAL MACHINE (JVM)

- . The key that allows Java to solve both the security and the portability problems is that the output of a Java compiler is not executable code. Rather, it is byte code.
- . Byte code is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM).
- . That is, in its standard form, the JVM is an interpreter for byte code.

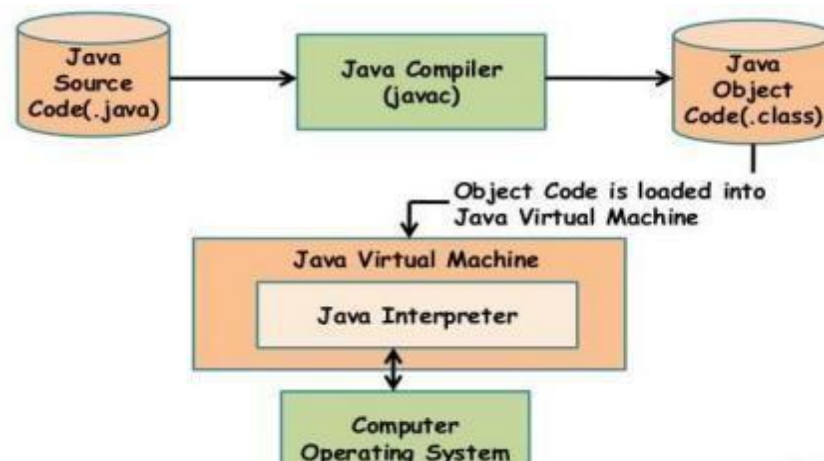


Fig:1.2.1 JVM

1.3 Buzz words or Features of JAVA:

The key considerations were summed up by the Java team in the following list of buzzwords/features:

- Simple.
- Secure.
- Portable.
- Object-oriented.
- Robust.
- Multithreaded.
- Architecture-neutral.
- Interpreted and High performance.
- Distributed.
- Dynamic.

1.4. Program Structure in Java

Introduction:

Java is an Object Orient and high-level programming language, originally developed by Sun Micro systems and released in 1995. Java runs on a variety of platforms, such as Windows, Mac OS, Linux etc. In Java, there are three types of programs as follows:

Stand-alone application programs: These programs are made and run on users' computers

Applet programs: These programs are meant to run in a web page on the Internet.

Java Servlets: These programs run in computers that provide web services. They are also often called server-side programs or servlets.

1.5. Writing Simple Java Programs

Java is an Object-oriented language. Every java program imports packages which are provides necessary classes and interfaces. For example:

```
import java.util.*;
```

```
import java.io.*;
```

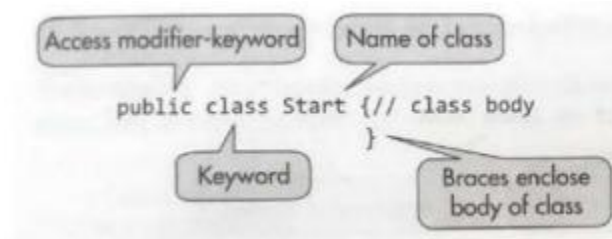
After import statement, every java program starts with the declaration of the class.

A program may have one or more classes.

A class declaration starts with the keyword class, followed by the identifier or name of the class. Giving the name of a package at the top is optional.

Class declaration contains name of the class and body of the class. The body of the class may consist of several statements and is enclosed between the braces {}.

A sample of class declarations is as follows



Fig, 1.5.1 Class Declaration

Here:

- . public is access specifier. This class is accessible to any outside code. Otherwise, the class is accessible to only same package classes.
 - . class is a keyword of java language which is used to declare the class.
 - . The class body starts with the left brace { and ends with the right closing brace }.
- // are comments which are neglected by the compiler.

A class body may comprise statements for declaration of variables, constants,

expressions, and methods.

Simple program in Java. Save this code in Text Document as “Start.java”

```
public class Start
{
    public static void main()
    {
        System.out.println("Hello World");
    }
}
```

Here:

class is a keyword

Start is a name of the class. Here class is declared as public, so it is available to all the classes.

main() is the method which initiate and terminate the program (in java functions are called as Methods)

println() is method of object “out”. “out” is the object of class “System”.

println() prints the string “Hello World” on the screen/monitor.

Compiling and Running Java program

Java compiler first converts the source code into an intermediate code, known as byte code or virtual machine code. To run the byte code, we need the Java Virtual Machine (JVM). JVM exists only inside the computer memory and runs on top of the Operating System. The byte code is not machine specific. The Java interpreter converts the byte code into Machine code. The following diagram illustrates the process of compiling and running Java programs.

For compiling the program, the Java compiler javac is run, specifying the name of the source file on the command line as depicted here:

javac Start.java

The Java compiler creates a file called Start.class containing the bytecode version of the program. The java interpreter in JVM executes the instructions contained in 4 this intermediate Java byte code version of the program. The Java interpreter is called with “java” at the command prompt.

C:\> java Start

Output:

Hello World

Here java command calls the Java interpreter which executes the Start.class (byte code of Start.java).

1.6 Elements or Tokens in Java Program

Java program contains different types of elements like white spaces, comments and tokens. A token is the smallest program element which is recognized by the compiler and which treats them as defined for the compiler. A program is a set of tokens which comprise the following elements:

Identifiers or names: Identifier is the name of variables, methods, classes etc.

Rules for framing Names or Identifiers.

- It should be a single word which contains alphabets a to z or A to Z, digits 0 to 9, underscore (_).
- It should not contain white spaces and special symbols.
- It should not be a keyword of Java.
- It should not be Boolean literal, that is, true or false.
- It should not be null literal.
- It should not start with a digit but it can start with an underscore.
- It can comprise one or more unicode characters which are characters as well as digits.

Conventions for Writing Names

- Names of packages are completely in lower-case letters such as mypackage, **java.lang**.
- Names of classes and interfaces start with an upper-case letter.
- Names of methods start with a lower-case character.
- Names of variables should start with a lower-case character.

Keywords: These are special words defined in Java and represent a set of instructions.

- The keywords represent groups of instructions for the compiler.
- These are special tokens that have a predefined meaning and their use is restricted.
- keywords cannot be used as names of variables, methods, classes, or packages.
- These are written in the lower case.
- Keywords of Java Language are as follows:

Table 1.6.1 List of Keywords

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

1.7 Literals: These are values represented by a set of character, digits, etc.

- A literal represents a value which may be of primitive type, String type, or null type. The value may be a number (either whole or decimal point number) or a sequence of characters which is called String literal, Boolean type, etc.
- A literal is a constant value.

1.8 Java Statements

A Statement is a instruction to the computer. A program is a set of statements or instructions. The statements specify the sequence of actions to be performed when some method or constructor is invoked. The statements are executed in the sequence in the specified order. The important Java statements are as follows.

Table 1.8.1: Java Statements

Statement	Description
Empty statement	These are used during program development.
Variable Declaration statement	It defines a variable that can be used to store the values.
Labeled statement	A block of statements is given a label. The labels should not be keywords, previously used labels, or already declared local variables.
Expression statement	Most of the statements come under this category. There are seven types of expression statements that include assignment, method call and allocation, pre-increment, post

	increment, pre-decrement, and post decrement statements.
Control statement	This comprises selection, iteration, and jump statements.
Selection statement	In these statements, one of the various control flows is selected when a certain condition expression is true. There are three types of selection statements including if, if-else, and switch.
Iteration statement	These involve the use of loops until some condition for the termination of loop is satisfied. There are three types of iteration statements that make use of while, do, and for.
Jump statement	In these statements, the control is transferred to the beginning or end of the current block or to a labeled statement. There are four types of Jump statements including break, continue, return, and throw.
Synchronization statement	These are used with multi-threading.
Guarding statement	These are used to carry out the code safely that may cause exceptions (such as division by zero, and so on). These statements make use of try and catch block, and finally

1.9 Command Line Arguments

A Java application can accept any number of arguments from the command line.

- These arguments can be passed at the time of running the program. This enables a programmer to check the behavior of a program for different values of inputs.
- When a Java application is invoked, the runtime system passes the command line arguments to the application's main method through an array of strings.
- It must be noted that the number of arguments in an array. To ensure this, we can make use of the length property of the array. This is illustrated in

Example 1

```
class vehicle
{
public static void main(String args[])
{
int x = args.length;
for(int i=0; i<x; i++)
{
System.out.println(args[i]);
}
}
```

Output

(After compiling, type the following lines on the command prompt. It produces the output as)

C:\> Java vehicle "Car Cycle Motorbike"

Car

Cycle

Motorbike

Example 2:

```
class Sum
{
public static void main(String args)
int s=0;
for(int i=0; i<args.length; i++){
s=s+Integer.parseInt(args[i]);

System.out.println(" The addition of passed arguments "+s );
}
```

Output

After compiling, the following lines are typed on the command

prompt: It produces the output as

C:\> Java Sum 20 15 17

Module-2

➤ **2.Data Types, Variables, and Operators:**

2.1 Introduction

Data Type is the type of the data which computer accepts. Every variable and expression has a data type that is known at the compile time.

- The declaration of data type with a variable or expression limits the types of values that a variable can have or the expression it can evaluate.
- Java is an object-oriented programming language based on classes and interfaces.
- Java defines some primitive (basic) data types that are not reference types of any class or interface. Eight primitive (basic) types of data are defined in Java. The type names are also the keywords shown here in bold letters

1. **Integral types**—byte, short, int, long

2. **Floating point types** - float, double

3. Character type -char

4. Boolean type – Boolean values – True, False

There is a non-data type called void and no data can be of type void. This type is used for methods that do not return any value. Java is a case-sensitive language. This means that it takes Area, area, and AREA are three different objects.

- Java defines eight primitive types of data: **byte, short, int, long, char, float, double, and Boolean.**
- The primitive types are also commonly referred to as simple types
- The primitive types represent single values—not complex objects.
- Although Java is otherwise object-oriented, the *primitive types are not*. They are analogous to the simple types found in most other non-object-oriented languages. The reason for this is efficiency. Making the primitive types into objects would have degraded performance too much.

2.2 Data Types in Java

The following chart summarizes the default values for all the above data types

Table 2.2.1: List of Data Types

Data Type	Default Value (for fields)
byte	0
short	0
Int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
String (or any object)	Null
boolean	False

1. Integers:

- Java defines four integer types: *byte, short, int, and long*. All of these are signed, positive and negative values.
- The width and ranges of these integer types vary widely, as shown in this table:

Table 2.2.2: Range of Integer Data Types

Name	Width	Range
Long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Int	32	–2,147,483,648 to 2,147,483,647

short	16	–32,768 to 32,767
byte	8	–128 to 127

a. byte:

- The smallest integer type is byte. This is a signed 8-bit type that has a range from –128 to 127.

Example: byte b, c;

b. short:

- Short is a signed 16-bit type.
- It has a range from –32,768 to 32,767. It is probably the least-used Java type.
- Examples:** short s, t;

c. int:

- It is a signed 32-bit type that has a range from –2,147,483,648 to 2,147,483,647.
- Examples:** int a, b =5;

d. long:

- long is a signed 64-bit type and is useful for those occasions where an int type is not large enough to hold the desired value.
- Examples:** long d, s;

1. Floating-Point Types:

- There are two kinds of floating-point types, *float* and *double*, which represent single- and double-precision numbers, respectively.

Table 2.2.3: Range of Floating Data Type

Name	Width in Bit	Approximate Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038

a. float:

- The type float specifies a single-precision value that uses 32 bits of storage.
- Examples:** float hightemp, lowtemp;

b. double:

- Double precision, as denoted by the double keyword, uses 64 bits to store a value.
- All transcendental math functions, such as sin(), cos(), and sqrt(), return double values.

2. Characters:

- It requires 16 bits. Thus, in *Java char is a 16-bit type*.
- The range of a char is 0 to 65,536. There are no negative chars.

- Although char is designed to hold Unicode characters, it can also be thought of as an integer type on which you can perform arithmetic operations.

3. **Booleans:**

- Java has a primitive type, called Boolean, for logical values.
- It can have only one of two possible values, true or false.
- This is the type returned by all relational operators, as in the case of $a < b$.
- Boolean variable requires one bit storage of memory.

2.3 VARIABLES:

- The variable is the basic unit of storage in a Java program.
- A variable is defined by the combination of an identifier, a type, and an optional initialize.
- In addition, all variables have a scope, which defines their visibility, and a lifetime.

Declaring a Variable:

- variable declaration is shown here:

data_type variable_name1=initial_value1, variable_name2=initial_value2;

- To declare more than one variable of the specified type, use a comma separated list.

Dynamic Initialization:

- Java allows variables to be initialized dynamically, using any expression valid at the time the variable is declared.

- **Example:**

```
class DynInit
{
    public static void main(String args[])
    {
        double a = 3.0, b = 4.0;
        double c = Math.sqrt(a * a + b * b);    // c is dynamically initialized
        System.out.println("Hypotenuse is " + c);
    }
}
```

- Here, three local variables—a, b, and c—are declared. The first two a and b, are initialized by constants. However, c is initialized dynamically to the length of the hypotenuse

2.4 CASTING

● **Type Conversion**

The process of converting one data type to another data type is called type conversion.

Generally, type casting is done in two ways:

1. **Implicit conversion (Automatic casting).**
2. **Explicit type casting.**

1. **Implicit conversion**: This implicit casting is automatically done by the java compiler. In this casting Source type memory must be larger than destination type.

Ex: long a; int b;

a=b; //implicit casting, because variable 'a' memory is larger than variable 'b' memory. This type of conversions is automatically done by the JAVA compiler.

2. **Explicit type casting**: This explicit type casting is not automatically done by the compiler; there is a separate syntax to perform explicit casting. In this casting source type variable memory is smaller than destination type variable memory. So there is a chance to get improper data (either MSBs or LSBs or discarded).

Syntax: (data_type)variable name;

Ex:

long a;

int b; b=(int)a; //explicit casting

2.5 The Scope and Lifetime of Variables:

- Java allows variables to be declared within any block.
- “A block is begun with an opening curly brace and ended by a closing curly brace”. A block defines a scope. Thus, each time you start a new block, you are creating a new scope.
- “A scope determines what objects are visible to other parts of your program”. It also determines the lifetime of those objects.
- The scope defined by a method begins with its opening curly brace. However, if that method has parameters, they too are included within the method's scope.
- As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.
- Thus, when you declare a variable within a scope, you are localizing that variable and protecting it from unauthorized access and/or modification.
- Scopes can be nested. When this occurs, the outer scope encloses the inner scope. This means that objects declared in the outer scope will be visible to code within the inner scope. However, the reverse is not true. Objects declared within the inner scope will not be visible outside it.
- Within a block, variables can be declared at any point, but are valid only after they are declared.

- Thus, if you define a variable at the start of a method, it is available to all of the code within that method.

The scope and lifetime of a variable is the part of the program in which it is visible and holds the last entered value In Java, there are distinctly two types of scopes.

(a) class scope and

(b) method scope

- A variable declared in a class has class scope and scope of a variable declared in a method has method scope.
- The variables declared in a block have block scope.
- Thus, the variables defined in main() at the beginning have scope in entire main() method, however, those defined in a block have block scope.
- A block starts with the left brace ({) and ends with the right brace (}). In the case of nested blocks of statements, the scope of variables is governed by the following rules.
 1. The scope starts from the point the variable is defined in the block (declared and value assigned to it).
 2. Although the variable may be defined anywhere in a block, it can be used only in the statements appearing after its definition Therefore, there is no use in defining a variable at the end of block.

If there are nested blocks, a variable defined in the outer block is visible in the inner block also and it cannot be redefined with the same name in the inner blocks

2.6 Literal Constants

- A constant value in Java is created by using a literal representation of it.
- For example, here are some literals:

100 98.6 'X' "This is a test"

- Left to right, the first literal specifies an integer, the next is a floating-point value, the third is a character constant, and the last is a string.
- A literal can be used anywhere a value of its type is allowed.

2.7 Symbolic Constants

A Symbolic constant is a variable whose value does not change throughout the program.

Some of the examples include PI, MAX, MIN, etc.

➤ 2.8 Introduction to Operators

An operator is a symbol that tells the computer to perform certain mathematical and logical calculations.

-The different types of Java operators are,

- a) Arithmetic operators
- b) Relational operators
- c) Logical operators
- d) Increment or decrement operators
- e) Assignment operators
- f) Conditional operators
- g) Bitwise operators
- h) Special operators

2.9 Precedence and Associativity of Operators :

If the expression contains several operators with the same precedence level, the expression is evaluated according to its *associativity*.

For example, in

$Z = 6 * 4 \% 5;$

Both the operators $*$ and $\%$ have same precedence level

Table 2.9.1: Precedence and Associativity of Operators

Category	Operator	Associativity
Primary	() [] -> . ++ -- new typeof sizeof checked unchecked default(T)	Left to Right
Unary	+ - ! ~ ++ -- (type)* & await	Right to Left
Multiplicative	* / %	Left to Right
Additive	+ -	Left to Right
Shift	<< >>	Left to Right
Relational and Type Testing	<<= >>= is as	Left to Right
Equality	== !=	Left to Right
Bitwise AND	&	Left to Right
Bitwise XOR	^	Left to Right
Bitwise OR		Left to Right
Logical AND	&&	Left to Right
Logical OR		Left to Right
Ternary	?:	Right to Left
Assignment and Lambda Expression	= += -= *= /= %= >>= <<= &= ^= = =>	Right to Left

Java supports the standard set of arithmetic, bitwise, relational, and logical operators

2.10 Basic Arithmetic Operators

- Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra.
- The following table lists the arithmetic operators:

Table 2.10.1: List of Arithmetic Operators

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus

- The operands of the arithmetic operators must be of a numeric type.

2.11 Bitwise Logical Operators

- Java defines several bitwise operators that can be applied to the integer types, long, int, short, char, and byte.
- These operators act upon the individual bits of their operands.
- They are summarized in the following table:

Table 2.11.1: List of Bitwise Operators

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
<<	Shift left
>>>	Shift right zero fill

2.12 Relational Operators or comparison operators:

- The relational operators determine the relationship that one operand has to the other.
- Specifically, they determine equality and ordering. The relational operators are shown here.

Table 2.12.1 : List of Relational Operators

Operator	Result	Operator	Result
==	Equal to	<	Less than
!=	Not equal to	>=	Greater than or equal to
>	Greater than	<=	Less than or equal to

- The outcome of these operations is a Boolean value.
- The relational operators are most frequently used in the expressions that control the if statement and the various loop statements.
- Any type in Java, including integers, floating-point numbers, characters, and Booleans can be compared using the equality test, `==`, and the inequality test, `!=`.

2.13 Boolean Logical Operators

The Boolean logical operators shown here operate only on Boolean operands. All of the binary logical operators combine two Boolean values to form a resultant Boolean value.

Table 2.15.1: List of Boolean Logical Operators

Operator	Result
!	Logical NOT
	Logical OR
&&	Logical AND

2.14 Short-Circuit Logical Operators:

- Java provides two interesting Boolean operators not found in many other computer languages. These are secondary versions of the Boolean AND (`&&`) and OR (`||`) operators, and are known as short-circuit logical operators.
- Java will not bother to evaluate the right-hand operand when the outcome of the expression can be determined by the left operand alone.

Examples:

(i) `if(0 == 1 && 2 + 2 == 4)`

```
{  
    out.println("This line won't be printed.");  
}
```

Java does the following:

1. Evaluate `0 == 1`, discovering that `0 == 1` is false.
2. Realize that the condition `(0 == 1 && whatever)` can't possibly be true, no matter what the whatever condition happens to be.
3. Return false (without bothering to check if `2 + 2 == 4`).

(ii) `if(2 + 2 == 4 || 0 == 1)`

```
{  
    out.println("This line will be printed.");  
}
```

Java does the following:

1. Evaluate `2 + 2 == 4`, discovering that `2 + 2 == 4` is true.
2. Realize that the condition `(2 + 2 == 4 || whatever)` must be true, no matter what the

whatever condition happens to be.

3. Return true (without bothering to check if `0 == 1`).

2.15 Increment (++) and Decrement (- -) Operators

- The increment operator is ++ which means +1
- The decrement operator is - - which means -1
- The operand must be either incremented or decremented by 1.

Example:

`m=5;y=++m;` results `m=6` and `y=6`.

`m=5;y=m++;` results `m=6` and `y=5`.

`m=5;y=--m;` results `m=4` and `y=4`.

`m=5;y=m--;` results `m=4` and `y=5`.

2.16 The Assignment Operator:

- The assignment operator is the single equal sign, =
- It has this general form:

`var = expression;`

Here, the type of var must be compatible with the type of expression.

- The assignment operator allows you to create a chain of assignments.
- For example, consider this fragment:

```
int x, y, z;  
x = y = z = 100; // set x, y, and z to 100
```

This fragment sets the variables x, y, and z to 100 using a single statement.

Compound Assignment operators:

`+=, -=, *=, /=, %=, &=, !=, ^=` and `<<=` etc...

2.17 The? Operator or Ternary operator or Conditional operator :

- Java includes a special ternary (three-way) operator that can replace certain types of if-then-else statements. This operator is the ?.
- General form:

`expression1? expression2: expression3`

- Here, expression1 can be any expression that evaluates to a Boolean value. If expression1 is true, then expression2 is evaluated; otherwise, expression3 is evaluated. The result of the? operation is that of the expression evaluated.
- Both expression2 and expression3 are required to return the same type, which can't be

void.

2.18 Special Operators.

- Java supports some special operators such as,

a) instanceof operator

b) member selection operator

a) instanceof operator:

- The instanceof is an object reference operator and returns true if the object on the left-hand side is an instance of the class given on the right-hand side.

- This operator allows us to determine whether the object belongs to a particular class or not.

Example:

person instanceof student

- It is true if the object person belongs to the class student, otherwise it is false.

b) member selection operator:

- The dot (.) operator is used to access the instance variables and methods of class objects.

Example:

person. Age // Reference to the variable age

person.Salary() // Reference to the method salary()

- It is also used to access classes and sub packages from a package.

Module-3

➤ 3. Control Statements:

3.1 Introduction:

A Control statement is a statement used to control the flow of execution in a Java Program.

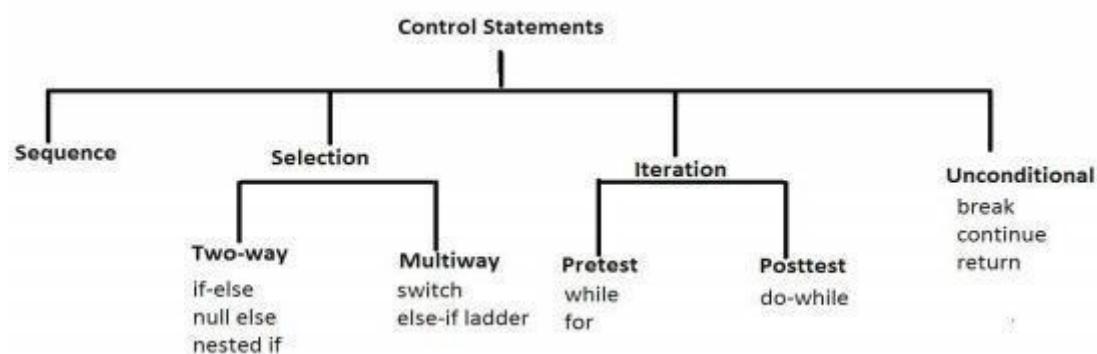


Fig 3.1.1: Structure of Control Statements

3.1.1 SELECTION STATEMENTS:

-Also called as conditional or decision-making control statements.

-There are two types in Selection control statements.

- i) Two-way selection control statements
- ii) Multi-way selection control statements

i) Two-way selection control statements:

-The different two-way selection statements are,

- a) if-else statement
- b) null else statement
- c) Nested-if statement

3.2 if Expression - also called as “ null else statement ”

Syntax:

```
if(condition)
{
```

statements;

}

next statement;

Example:

if(a==2)

{

p++;

}

System.out.println("program over");

3.3 Nested if Expressions

-if within if is called as Nested-if.

Syntax:

if(condition-1)

{

if(condition-2)

{

Statement-1;

}

else

{

Statement-2;

}

}

else

{

if(condition-3)

{

Statement-3;

}

else

{

Statement-4;

}

```
}
```

```
next statement;
```

Example:

```
if(a>b)
```

```
{
```

```
if(a>c)
```

```
{
```

```
System.out.println("a is greater");
```

```
}
```

```
else
```

```
{
```

```
System.out.println ("c is greater");
```

```
}
```

```
}
```

```
else
```

```
{
```

```
if(b>c)
```

```
{
```

```
System.out.println("b is greater");
```

```
}
```

```
else
```

```
{
```

```
System.out.println("c is greater");
```

```
}
```

```
}
```

3.4 if-else**Expressions Syntax:**

```
if(condition)
```

```
{
```

```
true-block statements;
```

```
}
```

```
else
```

```
{
```

```
false-block statements;
```

```
}
```

```
next statement;
```

Example:

```
if(a>b)
```

```
{
```

```
System.out.println("a is greater");
```

```
}
```

```
else
```

```
{
```

```
System.out.println("b is greater");
```

```
}
```

3.5 else-if ladder statement:**Syntax:**

```
if(condition-1)
```

```
{
```

```
Statement-1;
```

```
}
```

```
else if(condition-2)
```

```
{
```

```
Statement-2;
```

```
}
```

```
.....
```

```
.....
```

```
else if(condition n-1)
```

```
{
```

```
Statement-(n-1);
```

```
}
```

```
else
```

```
{
```

```
Statement-n;
```

```
}
```

```
next statement;
```

Example:

```
if(a>b&& a>c&&a>d)
```

```
{  
System.out.println("a is greater");  
}  
else if(b>a&&b>c&&b>d)  
{  
System.out.println("b is greater");  
}  
else if(c>a&&c>b&&c>d)  
{  
System.out.println("c is greater");  
}  
else  
{  
System.out.println("d is greater");  
}
```

3.6 Ternary Operator?:

In Java, the **ternary operator** is a type of Java conditional operator. The meaning of **ternary** is composed of three parts.

The **ternary operator** (**? :**) consists of three operands. It is used to evaluate Boolean expressions. The operator decides which value will be assigned to the variable. It is the only conditional operator that accepts three operands. It can be used instead of the if-else statement. It makes the code much more easy, readable,

and shorter.

Syntax:

Expression1 ? Expression2 : Expression3

- The first expression is the test condition.
- If it evaluates true, the Expression2 is executed; otherwise Expression3 is executed.

Example-1:

```
public class Ternary  
{  
public static void main (String args[])  
{
```

```
int a=10;
int b = (a<20)? 100 :200; // a < 20 if statement
System.out.println("b= "+b);

}
}
```

Output:

```
C:\>javac Ternary.java
```

```
C:\>java Ternary
```

```
b= 100
```

Example-2:

```
public class Ternary
{
    public static void main (String args[])
    {
        int a=10;
        int b = (a>20)? 100 :200; // a>20 – else statement
        System.out.println("b= "+b);
    }
}
```

Output:

```
C:\>javac Ternary.java
```

```
C:\>java Ternary
```

```
b= 200
```

3.7. Switch Statement**Syntax:**

```
switch(expression)
{
    case value-1:statement-1;break;
    case value-2:statement-2;break;
    .....
    .....
    case value-n:statement-n;break;
    default: default statement;
}
```

next statement;

Example:

```
switch(digit)
{
case 0: System.out.println("ZERO");break;
case 1: System.out.println("ONE");break;
case 2: System.out.println("TWO");break;
case 3: System.out.println("THREE");break;
case 4: System.out.println("FOUR");break;
case 5: System.out.println("FIVE");break;
case 6: System.out.println("SIX");break;
case 7: System.out.println("SEVEN");break;
case 8: System.out.println("EIGHT");break;
case 9: System.out.println("NINE");break;
default: System.out.println("Enter between 0-9");
}
```

3.8. Iteration Statements

LOOP STATEMENTS:

- The iteration control statements are also called as Repetition or Iteration control statements.
- A looping process includes the following four steps.
- Setting and initialization of a counter.
- Execution of the statements in the loop body.
- Test for a specified condition (loop control expression) for execution of a loop
- Incrementing or Decrementing counter.

i) Pretest and Posttest loops:

- In a **Pretest loop**, the condition is checked before we execute a loop body.
- It is also called as **entry-controlled loop**.
- In the **Posttest loop**, we always execute the loop body atleast once.
- It is also called as **exit-controlled loop**.

3.9. while Expression

a) while statement:

Syntax:

while(condition)


```
{  
loop body;  
}  
  
next statement;  
Example:  
n=10,i= 1,sum=0;  
while(i<=n)  
{  
sum=sum+i;  
i++;  
}  
System.out.println("sum="+sum);
```

3.10 do-while Loop - for

Loop Syntax:

```
do  
{  
loop body;  
}while(condition);  
next statement;  
  
Example:  
n=10,i= 1,sum=0;  
do  
{  
sum=sum+i;  
i++;  
} while(i<=n);  
System.out.println("sum="+sum);
```

3.11 for Loop

for statement:

Syntax:

for(initialization;condition;inc or dec)

{

loop body;

}

next statement;

Example:

n=10,i,sum=0;

for(i= 1,j=1;j<=n;j++,i+=2)

{

sum=sum+i;

}

System.out.println("sum="+sum);

3.12 Nested for

Loop Syntax:

for(initialization;condition;inc or dec)

{

for(initialization;condition;inc or dec)

{

Inner loop body;

}

Outer loop body;

}

next statement;

Example:

n=10,i, j, sum=0;

for(i= 1;i<=n;i++)

{

sum=sum+i;

}

```
System.out.println("sum="+sum);
```

3.13 For-Each Loop

The Java for-each loop or enhanced for loop. It provides an alternative approach to traverse the array or collection in Java. It is mainly used to traverse the array or collection elements.

The advantage of the for-each loop is that it eliminates the possibility of bugs and makes the code more readable. It is known as the for-each loop because it traverses each element one by one.

Advantages:

1. Less clutter in code, especially when iterators are used.
2. Less chances of errors.
3. Improves overall readability of program.

Limitations:

1. It is designed to iterate in forward direction only.
2. In iteration, it takes a single step at a time.
3. It cannot simultaneously traverse multiple arrays or collections.

Syntax:

for (Object obj : Collection_name)

{

Body of loop

}

Example:

```
public class ForEach
{
    public static void main (String args[])
    {
        int myArray[] = {10,20,30,40,50};
        for (int x:myArray)
        {
            System.out.println(" "+x);
        }
    }
}
```

Output:

```
C:\>javac ForEach.java
```

```
C:\>java ForEach
```

```
10
```

```
20
```

```
30
```

```
40
```

```
50
```

3.14 Break Statement

Unconditional control

statements:

-The unconditional control statements are,

- a) break statement
- b) continue statement

break statement:

-The break statement skips from the loop or block in which it is defined.

-The control then automatically goes to the first statement after the loop or block.

-The general format is

break;

Example:

```
public class Break
{
    public static void main (String args[])
    {
        //printing the values from 1 to 10
        for(int i =0; i<10;i++)
        {
            if(i==5)
            break;
            System.out.println( i );
        }
    }
}
```

Output:

```
C:\>javac Break.java
```

```
C:\>java Break
```

```
0
```

```
1
```

```
2
```

```
3
```

```
4
```

Here loop is stopped due to break statement.

3.15 Continue Statement.

-The continue statement is used for continuing next iteration of loop statements. When it occurs in the loop, it does not terminate but it skips the statements after it. It is useful when we want to continue the program without executing any part of the program.

-The general format is

continue;

Example:

```
public class Continue
{
public static void main (String args[])
{
//printing the values from 1 to 10
for(int i =0; i<10;i++)
{
if(i==5)

continue; // Continue statement
System.out.println( i );
}
}
}
```

Output:

```
C:\>javac Break.java
```

```
C:\>java Continue
```

```
0
```

1
2
3
4
6
7
8
9

Here 5 is not printed because of continue statement. The Iteration at the condition $i == 5$ is skipped or jumped to next statement.

Assignment cum Tutorial Questions**Part-A****Objective Questions (1M)****MODULE 1: Object Oriented Programming**

1. Java language was initially called as _____
2. Java programs are _____ []
(a) Compiled (b) Interpreted (c) Both Compiled & Interpreted (d) None of these
3. The output of a Java Compiler is _____ file []
(a) Byte code (b) .class file (c) Both a and b (d) .exe file
4. JVM stands for Java Virtual Machine
5. Java .class files are Platform Independent (True / False) Ans:
6. Which of the following Command is used to compile java program: []
(a) javac (b) .java (c) .run (d) .execute
7. What is the role of JIT compiler? []
a) To convert .class file into managed code.
b) To convert .java file into managed code.
c) To convert .class file into un-managed code.
d) To convert .java file into un-managed code.
8. What is the role of JRE? []
a) Used to handle java libraries.
b) Same function as equal to JVM.
c) Used to convert java source code into machine code.
d) None of these.
9. Which of the following OOP feature is related to Coupling and Cohesion? []
a) Polymorphism (b) Inheritance (c) Data Encapsulation (d) Dynamic Binding.
10. Which of the following language is an example of Object based language? []
a) Ada (b) VB Script (c) Java Script (d) all of the above
11. Which one of the following is/are an example of pure OOP language []
a) Smalltalk, Ruby, Scala, Python (b) Eiffel
c) java (d) both a and b
12. Which one of the OOP features is related to Late Binding []
a) Inheritance (b) Polymorphism (c) Data Encapsulation (d) Abstraction

13. Java is Platform Independent because of []
a) Byte code b) Interpreter c) Compiler d) JIT
14. Code re-usability in OOP is related to []
a) Polymorphism b) Inheritance
c) Data Encapsulation d) Data abstraction
15. Match the following: []
A) Inheritance 1) Virtual functions
B) Polymorphism 2) Wrapping up of data and functions into single unit
C) Encapsulation 3) Code re-usability
D) Abstraction 4) Data Hiding
a) A-1, B-2, C-3, D-4 b) A-3, B-1, C-4, D-2
c) A-3, B-1, C-2, D-4 d) A-4, B-3, C-2, D-1
16. Match the following: []
A) JVM 1) set of libraries and classes
B) JRE 2) Software development environment
C) JDK 3) Running of Java Programs
D) JIT 4) used to translate Byte code to Machine code.
a) A-3, B-1, C-4, D-2 b) A-3, B-1, C-2, D-4
c) A-4, B-2, C-1, D-4 d) A-3, B-2, C-1, D-4

Module 2: Data Types, Variables, and Operators

1. What is auto-boxing []
a) Converting primitive types into class type.
b) Converting object type into class type.
c) Converting primitive type into primitive type.
d) Converting class type into class type.
2. What is the need of Wrapper classes in JAVA? []
a) They are used to convert primitive type to class type.
b) They are used to convert class type to primitive type.
c) They are used to convert string type to primitive type.
d) All of the above.

3. Which of the following syntax is suitable to represent 'main()' function Java []
- a) public static void main(String[] args)
 - b) static public void main(String args[])
 - c) static public void main(String... args)
 - d) public static void main(String... args)
 - e) All of the above
4. Which of the following is valid output statement in JAVA (Ex: a=10, b=20) []
- a) System.out.println("Additoon of"+a+" and "+b+" is:"+(a+b));
 - b) System.out.printf("Additon of %d and %d is: %d", (a+b));
 - c) System.out.print("Addition of "+a+ " and "+b+"is" +(a+b));
 - d) All of the above.
5. What is the output of the following Command in JAVA: []
- ```
Sytem.out.println(Character.SIZE);
```
- a) 16                      b) 2                      c) Compile time error                      d) Garbage value
6. Match the following: [       ]
- |                       |                                                  |
|-----------------------|--------------------------------------------------|
| A) Integer.parseInt() | 1) to pass any number of arguments without array |
| B) class              | 2) auto-boxing                                   |
| C) Integer n=10       | 3) Wrapper class method                          |
| D) varargs            | 4) user defined data type.                       |
- 
- |                       |                       |
|-----------------------|-----------------------|
| a) A-1, B-2, C-3 ,D-4 | b) A-3, B-4, C-2, D-1 |
| c) A-3, B-2, C-4, D-1 | d) A-4, B-2, C-1, D-3 |

### Module 3: Control Statements

1. Java does not support goto label, pointers and global variables [True/False] Ans:
2. It is possible to place string as switch-case label in JAVA? [True/False] Ans:
3. Java does not support Jagged Arrays [True/False] Ans:
4. What is the output of the following code: [       ]  

```
class GEC
{
 static public void main(String args[])
 {
 int[] a;
 a=new int[5];
```

```
for(int x:a)
```

```
System.out.print(x);
```

```
}}
```

a) Garbage values    b) Compile time error    c) 00000    d) 11111

5. What is the output of the following code:

[       ]

```
class GEC
```

```
{
```

```
static public void main(String args[])
```

```
{
```

```
int[][]a=new int[3][];
```

```
a[0]=new int[5];
```

```
a[1]=new int[6];
```

```
a[2]=new int[7];
```

```
for(int i=0;i<3;i++)
```

```
for(intj=0;j<5;j++)
```

```
System.out.println(a[i][j]);
```

```
}}
```

- a) Compile time error                      b) It is jagged array produce 15 zeros as output.  
c) Garbage Values                          d) None of these.

## Part-B

### Short Answer Questions (2M)

#### Module 1: Object Oriented Programming

- 1) Explain briefly about features of OOPs? [BL2]
- 2) Explain about Buzz words of JAVA? [BL2]
- 3) What is literal in JAVA explain with an example? [BL2]
- 4) What is the difference between static binding and dynamic binding? [BL3]
- 5) What are the applications of OOPs? [BL2]

**Module 2: Data Types, Variables, and Operators**

1. Distinguish between type conversion and type casting. [BL3]
2. Explain about Implicit and explicit type casting in JAVA? [BL3]
3. Write about primitive data types in java with suitable examples to each type. [BL2]
4. State and explain the rules to frame an identifier in java. List any 4 valid identifiers and 4 invalid identifiers. [BL3]
5. What is boxing and un-boxing (Casting) in JAVA? [BL2]
6. Explain Scope and life time of variable in JAVA? [BL2]

**Module 3: Control Statements**

1. Write the syntax for Nested if else with an example. [BL2]
2. Differentiate While and Do-while conditions. [BL3]
3. Explain the break and continue with an example. [BL2]
4. Explain the importance of break statements in switch. [BL3]

## **Part-C**

### **Descriptive Questions (5M)**

**Module 1: Object Oriented Programming**

1. Write the importance of Java Virtual Machine. [BL2]
2. Explain the basic characteristics of object oriented programming. [BL2]
3. Outline the role of JVM in making Java Platform independent. [BL3]
4. List and explain the Principles of OOP programming languages. [BL2]
5. Define Bytecode. Explain the role of JVM in executing a Java program [BL3]
6. Differentiate Procedure Oriented Programming and Object Oriented Programming. [BL3]
7. Describe the features of Java. [BL2]
8. Create a JAVA program by using Command line arguments? [BL3]

**Module 2: Data Types, Variables, and Operators**

- 1) Explain about Primitive data types in Java? [BL2]
- 2) Explain about user defined data types in Java? [BL2]
- 3) What are Wrapper classes in Java and explain some methods which are present in Wrapper classes? [BL2]

- 4) Explain about Implicit and explicit type casting in JAVA? [BL3]
- 5) Write a Java Program to apply Type Conversion on a suitable case study [BL3]
- 6) Develop a Java program to find smallest of three numbers using ternary operator.[BL3]
- 7) Write a java program that demonstrates the Boolean logical operators. [BL3]
- 8) Develop a java program to calculate the product of 2 numbers without using \* operator. [BL3]

### Module 3: Control Statements

1. Develop JAVA program to find out the reverse of individual array element and display the array using advanced or enhanced 'for' loop. [BL3]
2. Write a program in java to display whether the entered number is even or odd.[BL3]
3. Write a Java program that calculates the product of the odd integers from 1 to 15. [BL3]
4. Develop Java code to check given number is Palindrome or not. [BL3]
5. Write a JAVA program to find out whether the given number is Armstrong or not using 'for' loop? [BL3]
6. Write a JAVA Program to display Prime series up to the given number? [BL3]
7. Write a java program to find factorial of a given number. [BL3]
8. Develop JAVA program to illustrate enhanced 'for' loop? [BL3]

### Part-D

### Competitive Examination Questions

1. What is the output of the following code: [       ]

```
class A
{
static public void main(String[] args)
{
int a=10, b=20;

System.out.println(b+++a+++b);
}}
```

a) Compile time error    b) 50            c) 51        d) 52

2. What is the output of the following code: [       ]

```
class A
{
static public void main(String[] args)
{
System.out.println(10.5%5);
}}
```

- a) 0.0                      b) 0.5                      c) % not applicable to floating values                      d) 2.1

**UNIT- 2****Classes and Objects & Methods****Modules**

1. Classes and Objects.
2. Methods.

**Objective:**

- . Develop the code with the concepts of Class, objects and its methods.

**Syllabus:**

Introduction, Class Declaration and Modifiers, Class Members, Declaration of Class Objects, Constructor Methods for Class, Keyword this. Methods: Introduction, Defining Methods, Overloaded Methods, Attributes Final and Static.

**Classes and Objects:** Introduction, Class Declaration and Modifiers, Class Members, Declaration of Class Objects, Assigning One Object to Another, Accessing Private Members of Class, Constructor Methods for Class, Overloaded Constructor Methods, Nested Classes, Keyword this.

**Methods:** Introduction, Defining Methods, Overloaded Methods, Passing Arguments by Value and by Reference, Class Objects as Parameters in Methods, Recursive Methods, Nesting of Methods, Attributes Final and Static.

**Learning Outcomes:**

Students will be able to

- . describe how classes, objects, methods and constructors are created and applied in java.
- . demonstrate how the objects communication takes place in java.
- . build java programs by applying oops concepts.
- . illustrate various parameter passing techniques.
- . implement method overloading .

## 1. Module 1: Classes and Objects.

### 1.1 Class Fundamentals:

- . A class defines a new data type. Once defined, this new type can be used to create objects of that type.
- . A class is a *template* for an object, and an object is an *instance* of a class. Because an object is an instance of a class, you will often see the two words *object* and *instance* used interchangeably.
- . A class is declared by use of the class keyword.

. Declaration of class:

```
class classname
{
 type instance-variable1;
 type instance-variable2;
 // ...
 type instance-variableN;
 type methodname1(parameter-list)
 {
 // body of method
 }
 type methodname2(parameter-list)
 {
 // body of method
 }
 // ...
 type methodnameN(parameter-list)
 {
 // body of method
 }
}
```

- . The data, or variables, defined within a class are called *instance variables*.
- . The code is contained within *methods*. Collectively, the methods and variables defined within a class are called *members* of the class.
- . The methods defined in the class are accessible using the instance variables.

- . Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables.
- . The data for one object is separate and unique from the data for another.

**Example of class:**

- . Create a class structure that may represent the structure of a hospital or other medical organization, begin with a class called Employee.
- . An employee has several characteristics that you can represent as variables, such as name, salary, and sickDays.
- . Write a method details() which consists of three println() statements that print the values of instance variables. The Employee class can be defined as follows:

```
class Employee
{
 String name; // Instance Variables
 int salary; // Instance Variables
 void details() // Instance Method
 {
 System.out.println("Name: " + name);
 System.out.println("Salary: " + salary);
 }
}
```

**1.2 Class modifiers:**

- . Class modifiers are used to control the access to class and its inheritance characteristics.
- . Java consists of packages and the packages consist of sub-packages and classes
- . Packages can also be used to control the accessibility of a class
- . These modifiers can be grouped as (a) access modifiers and (b) non-access modifiers.
- . Table 1.1 gives a description of the various class modifiers.

| Modifiers               | Description                                                                                                                                       |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Access Modifiers</b> |                                                                                                                                                   |
| <b>default</b>          | The class declared without an access modifier is accessible to the class in its own package only. It is not visible to classes in other packages. |
| <b>public</b>           | A Class declared public is accessible to all classes in all packages.                                                                             |
| <b>private</b>          | A Class specified as private has the highest degree of protection and is                                                                          |



|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | accessible to other members of outer(nesting) classes.                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>protected</b>            | The class is accessible to other members of outer(nesting) class as well as by classes derived from the outer class.                                                                                                                                                                                                                                                                                                                                        |
| <b>Non-access Modifiers</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>final</b>                | A class declared final cannot have sub(derived) classes.                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>abstract</b>             | A class declared as abstract must have one or more abstract methods as its members or its super class contains an abstract method that is not fully defined in the present class.                                                                                                                                                                                                                                                                           |
| <b>strictfp</b>             | The modifier makes all the Root and double expressions declared in the class and in its nested classes to be FP-strict expression, all intermediate values are either float value set or the double value set. This implies that the same result is obtained if the operations involving floating point variable is performed in any platform. The results of FP-strict expressions must be supported by IEEE 754 arithmetic for float and double operands. |

Table 1.1. Summary of class modifiers.

Examples:**1. A class without modifier.**

```
class Student
{
 /* class body*/
}
```

**2. A class with modifier**

```
public class Student
{
 /* class body*/
}
```

(or)

```
private class Student
{
 /* class body*/
}
```

(or)

```
protected class Student
```

```
{
 /* class body*/
}
(or)
final class Student
{
 /* class body*/
}
(or)
abstract class Student
{
 /* class body*/
}
```

### 1.3 Class Members

- . The class members are declared in the body of a class. These may comprise fields (variables in a class), methods, nested classes, and interfaces.
- . The members of a class comprise the members declared in the class as well as the members inherited from a super class. The scope of all the members extends to the entire class body.
- . The fields comprise two types of variables
  1. **Non Static variables** : These include *instance* and *local variables* and varies in scope and value.
    - **Instance variables:** These variables are individual to an object and an object keeps a copy of these variables in its memory.
    - **Local variables:** These are local in scope and not accessible outside their scope.
  2. **Class variables ( Static Variables)** : These variables are also qualified as static variables.
- . The values of these variables are common to all the objects of the class. **The class keeps only one copy of these variables and all the objects share the same copy.** As class variables belong to the whole class, these are also called class variables.

**Example:** Java Program to demonstrate Class Members.

```
class CustomerId
```

```
{
 static int count=0; // static variable

 int id; // instance variable
 CustomerId() // Constructor
 {
 count++;
 id = count ;
 }

 int getId() // Method
 {
 return id;
 }

 int localVar()
 {
 int a=10; // Local variable
 return a;
 }
}

public class Application
{
 public static void main(String[] args)
 {
 CustomerId obj = new CustomerId();
 System.out.println("Customer Id = " + obj.getId());
 System.out.println("Local Variable = " + obj.localVar());
 }
}
```

**Output:**

C:\>javac Application.java

C:\>java Application

Customer Id = 1

Local Variable = 10

**1.4 Declaration of Class Objects:**

- . Object is an instance of a class.

(or)

- . Objects are basic run-time entities in an object-oriented system.

(or)

- . Any real-world entity is called an object.

(or)

- . Objects are the combination of data and methods.  
Example: Person, Place, bank account, ....., so on.
  - In the real-world only objects are visible but classes are invisible.
  - The most important benefits of an objects are
    - Modularity
    - Reusability
  - The properties of objects are two types
    - visible
    - invisible
- . Let man is an object, then visible properties are eyes, ears, hands, legs,...so on and invisible properties are name, blood group,.... so on.
- . - Every object contains three basic elements
  - Identity (name)
  - State (variables)
  - Behaviour (methods)
- . Obtaining objects of a class is a two-step process.
- . First, we must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can *refer* to an object.
- . Second, we must acquire an actual, physical copy of the object and assign it to that variable. We can do this by using the new operator.
- . The new operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new. This reference is then stored in the variable.

#### **Syntax for creating an Object:**

```
Classname objectname=new Classname();
```

#### **Example:**

```
Employee sam=new Employee();
```

This statement combines the two steps just described. It can be rewritten like this to show each step more clearly:

```
Employee sam; // declare reference to object
```

```
sam=new Employee(); // allocate a Employee object
```

The first line declares *sam* as a reference to an object of type Employee.

- After this line executes, *sam* contains the value null, which indicates that it does not yet point to an actual object.
- Any attempt to use *sam* at this point will result in a compile-time error. The next line allocates an actual object and assigns a reference to it to *sam*.
- After the second line executes, you can use *sam* as if it were an Employee object. But in reality, *sam* simply holds the memory address of the actual Employee object.
- The effect of these two lines of code is depicted in Figure 1.1:

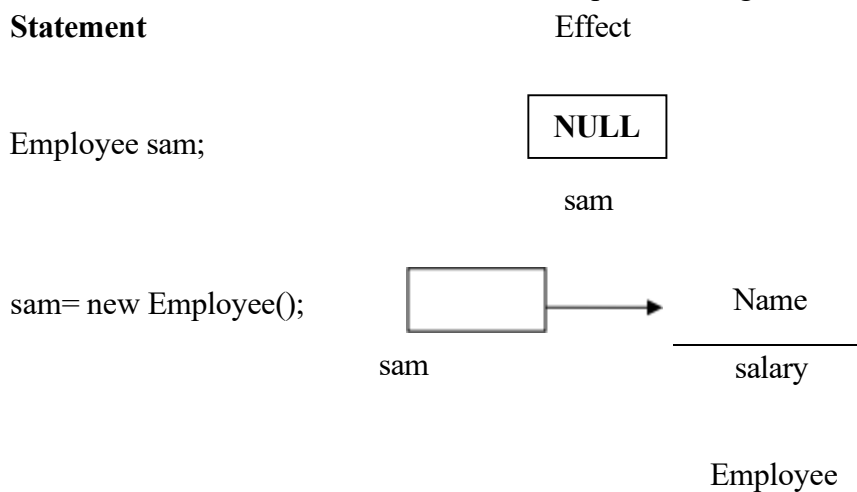


Fig 1.1: Declaring a Object type sam

Example: Here is a complete program that uses the Employee class:

```
class Employee
{
 String name; // person's name // Instance Variables
 double salary; // salary in dollars
 void details() // Instance Method
 {
 System.out.println("Name: " + name);
 System.out.println("Salary: " + salary);
 }
}

public class MainProgram
{
 public static void main(String[] args)
```

```
{
 Employee ram = new Employee(); //Object declaration
 ram.name = "Ram";
 ram.salary = 32000;
 ram.details();
}
}
```

The output produced by this program is shown here:

Name:Ram

Salary:32000

### **new Operator:**

- . The new operator dynamically allocates memory for an object. It has this general form:  

*class-var* = new *classname*( );
- . *class-var* is a variable of the class type being created.
- . The *classname* is the name of the class that is being instantiated. The class name followed by parentheses specifies the *constructor* for the class.
- . A constructor defines what occurs when an object of a class is created.
- . Most real-world classes explicitly define their own constructors within their class definition. However, if no explicit constructor is specified, then Java will automatically supply a default constructor.
- . This is the case with Employee. For now, we will use the default constructor.
- . At this point, you might be wondering why you do not need to use new for such things as integers or characters. The answer is Java's primitive types are not implemented as objects.
- . It is important to understand that new allocates memory for an object during run time.
- . The advantage of this approach is that your program can create as many or as few objects as it needs during the execution.

### **1.5 Constructor Methods for Class**

- . Java allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor.
- . A **constructor** is a special method which initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically

similar to a method.

- . Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes.
- . Constructors look a little strange because they have no return type, not even void. This is because the implicit return type of a class' constructor is the class type itself.
- . It is the constructor's job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.
- . Let's begin by defining a simple constructor that simply sets the dimensions of each Box to the same values.

**Example:** Java program to demonstrate default constructor.

```
class Box
{
 double width;
 double height;
 double depth;
 Box() // This is the constructor for Box.
 {
 System.out.println("Constructing Box");
 width = 10;
 height = 10;
 depth = 10;
 }
 double volume()
 {
 // compute and return volume
 return width * height * depth;
 }
}
```

```
public class BoxDemo6
{
 public static void main(String args[])
 {
 // declare, allocate, and initialize Box objects
 Box mybox1 = new Box();
 Box mybox2 = new Box();
 double vol;

 vol = mybox1.volume(); // get volume of first box
 System.out.println("Volume is " + vol);

 vol = mybox2.volume(); // get volume of second box
 System.out.println("Volume is " + vol);

 }
}
```

When this program is run, it generates the following results:

Constructing Box

Constructing Box

Volume is 1000.0

Volume is 1000.0

They initialize the details of employee to both mybox1 and mybox2. Most constructors will not display anything. They will simply initialize an object.

- let's re examine the new operator. As you know, when you allocate an object, you use the following general form:

*class-var* = new *classname*( );

- Now you can understand why the parentheses are needed after the class name. What is actually happening is that the constructor for the class is being called. Thus, in the line

Box mybox1 = new Box();

- new Box( ) is calling the Box( ) constructor. When we do not explicitly define a constructor for a class, then Java creates a default constructor for the class.
- This is why the preceding line of code worked in earlier versions of Employee that did not define a constructor.
- The **default constructor** automatically initializes all instance variables to zero.
- Once we define our own constructor, the default constructor is no longer used.



**Parameterized Constructors:**

- . While the Box( ) constructor in the preceding example does initialize a Box object, it is not very useful—all boxes have the same dimensions.
- . The easy solution is to add parameters to the constructor. This makes them much more useful.

**Example:** Java Program to demonstrate parameterized constructor to initialize the dimensions of a box.

```
class Box
{
 double width;
 double height;
 double depth;

 Box(double w, double h, double d)//This is the constructor for Box
 {
 width = w;
 height = h;
 depth = d;
 }

 double volume()
 {
 // compute and return volume
 return width * height * depth;
 }
}

public class BoxDemo7
{
 public static void main(String args[])
 {
 // declare, allocate, and initialize Box objects
 Box mybox1 = new Box(10, 20, 15);
 Box mybox2 = new Box(3, 6, 9);
 Box mybox3 = new Box();
 double vol;

 vol = mybox1.volume(); //// get volume of first box
 System.out.println("Volume is " + vol);
 }
}
```

```
 vol = mybox2.volume(); // get volume of second box
 System.out.println("Volume is " + vol);
 }
}
```

The output from this program is shown here:

Volume is 3000.0

Volume is 162.0

- . As you can see, each object is initialized as specified in the parameters to its constructor.

For example, in the following line,

```
Box mybox1 = new Box(10, 20, 15);
```

- . The values 10, 20, and 15 are passed to the Box( ) constructor when new creates the object. Thus, mybox1's copy of width, height, and depth will contain the values 10, 20, and 15, respectively.

## 1.6 Overloaded Constructor Methods

- . The name of all the overloaded constructor methods same as the name of the class, but parameters have to be different either in number of type a order of parameters in each definition.

**Example:** Java program to demonstrate overloading constructors.

```
class Box
```

```
{
 double width;
 double height;
 double depth;
 Box(double w, double h, double d)
 {
 //constructor used when all dimensions specified
 width = w;
 height = h;
 depth = d;
 }
 Box()
}
```

```
 {
 // constructor used when no dimensions specified
 width = -1; // use -1 to indicate
 height = -1; // an uninitialized
 depth = -1; // box
 }
Box(double len)
{
 // constructor used when cube is created
 width = height = depth = len;
}
double volume()
{
 // compute and return volume
 return width * height * depth;
}
}

public class OverloadCons
{
 public static void main(String args[])
 {
 // create boxes using the various constructors
 Box mybox1 = new Box(10, 20, 15);
 Box mybox2 = new Box();
 Box mycube = new Box();
 double vol;

 vol = mybox1.volume(); // get volume of first box
 System.out.println("Volume of mybox1 is " + vol);
 vol = mybox2.volume(); // get volume of second box
 System.out.println("Volume of mybox2 is " + vol);
 vol = mycube.volume(); // get volume of cube
 System.out.println("Volume of mycube is " + vol);
 }
}
```

**Output:**

Volume of mybox1 is 3000.0

Volume of mybox2 is -1.0.

Volume of mycube is 343.0

- . As you can see, the proper overloaded constructor is called based upon the parameters specified when **new** is executed.

### 1.7 Keyword this.

- . Java defines the 'this' keyword. this can be used inside any method to refer to the *current* object.
- . this is always a reference to the object on which the method was invoked.
- . We can use this anywhere a reference to an object of the current class' type is permitted.

**Example: Java Program to demonstrate the usage of this.**

```
class Add
{
 int a,b;
 void setValues(int a, int b)
 {
 this.a = a;
 this.b = b;
 }
 void add()
 {
 System.out.println("Sum = "+ (a+b));
 }
}

class ThisKeyword
{
 public static void main (String args[])
 {
 Add obj= new Add();
 obj.setValues(10,20);
 obj.add();
 }
}
```

Output:

```
C:\>javac ThisKeyword.java
```

```
C:\>java ThisKeyword
```

```
Sum = 30
```

## 2. Module 2: Methods.

### 2.1 Introduction

- . A method in Java represents an action on data or behavior of an object. In other programming languages, the methods are called functions or procedures.
- . A method is an encapsulation of declarations and executable statements meant to execute desired operations.
- . A few types of actions and behavior of Methods are as follows.
  - It could involve carrying out computation on data presented to method.
  - The action may simply be rearranging the elements of an object. for example, sorting arrays.
  - The action may comprise finding or searching elements in the list.
  - The action may simply be the initialization of an object.
  - Methods may create images, voice, and multimedia as well as display.
  - Methods may define how an object will communicate with other objects.
  - It may simply answer an enquiry.
  - A method may tell whether an action is permissible or not.
- . In Java, a method must be defined inside a class and an interface.
- . An interface represents an encapsulation of constants, classes, interfaces, and one or more abstract methods that are implemented by a class.
- . A method cannot be defined inside another method, but it can be defined inside a local class.

## 2.2 Defining Methods

- . A method definition comprises two components:
- . Header that includes modifier, type, identifier, or name of method and a list of parameters. The parameter list is placed in a pair of parentheses.
- . Body that is placed in braces ({ }) and consists of declarations and executable statement and other expressions.

### Method definition:

Modifier return\_type method\_name (datatype Parameter\_Name,...)

{

/\*Statements --

Body of the method\*/

}

- . The following Figure 1.2 shows description of non-access modifiers.

Brief description of non-access modifiers

| Non-access modifiers | Description                                                                                                                                                                                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| static               | With this modifier, a method may be called without an object of class. However, class reference is needed.                                                                                                                                                         |
| final                | Method declared final cannot be modified (overridden) in a subclass.                                                                                                                                                                                               |
| native               | It is used only for methods. It indicates that the method is implemented in a platform-dependent language like C.                                                                                                                                                  |
| transient            | A variable is declared transient if it is desired that it should not be part of the persistent state of the object, that is, the transient variable value is not stored, whereas non-transient variables are stored when an object is stored in persistent memory. |
| synchronized         | It is applied to make method thread safe. When it is used, it ensures that the method can be accessed by only one thread at a time.                                                                                                                                |
| volatile             | It indicates to the compiler that a variable can be modified by the other thread.                                                                                                                                                                                  |

Figure 1.2 Description of non-access modifiers.

**Example:** Java Program to create methods.

```
class Add
```

```
{

 int a,b;

 void setValues(int x, int y) // method with two arguments

 {

 a = x;

 b = y;

 }

 void add() // method without arguments

 {

 System.out.println("Sum = "+ (a+b));

 }

}

class MethodDemo

{

 public static void main (String args[])

 {

 Add obj= new Add();

 obj.setValues(10,20); // method calling

 obj.add();

 }

}
```

**Output:**

C:\>javac MethodDemo.java

C:\>java MethodDemo

Sum = 30

## 2.3 Overloaded Methods

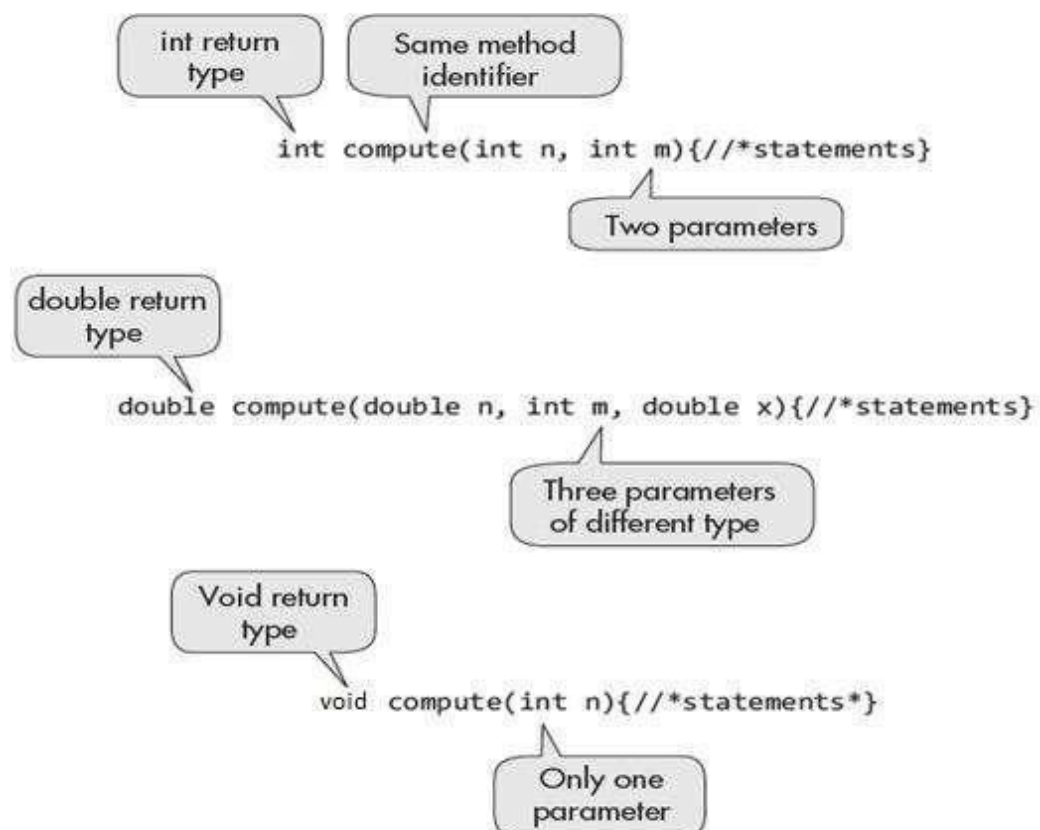
- Methods with the same name and scope are permitted provided they have different signatures that include the following:

### i. Number of parameters

### ii. Data types of parameters

### iii. Their order in the parameter list

- The compiler executes the version of the method whose parameters match with the arguments.
- For example, the following types of declarations in the scope are permissible shown in Figure 1.3.





**Example: Java program to demonstrate method overloading.**

```
class Add

{

 int a,b;

 void setValues(int a, int b) // method with two arguments

 {

 this.a = a;

 this.b = b;

 }

 void add() // method without arguments

 {

 System.out.println("In add() method Sum = "+ (a+b));

 }

 //Method overloading - integer datatype arguments

 void add(int a, int b)

 {

 System.out.println("In add(int, int) Method- sum= "+ (a+b));

 }

 //Method overloading - double datatype arguments

 void add(double a, double b)

 {

 System.out.println("In add(double, double) MethodSum = "+ (a+b));

 }

}
```

```
}

public class MethodOverload
{

 public static void main (String args[])
 {

 Add obj= new Add();

 obj.setValues(10,20); // method calling

 obj.add(); // calling method without arguments

 obj.add(15,30); // calling method with integer datatype arguments

 obj.add(10.3, 30.4); // calling method with double datatype arguments

 }

}
```

**Output:**

C:\>javac MethodOverload.java

C:\>java MethodOverload

In add() method Sum = 30

In add(int, int) Method- sum= 45

In add(double, double) MethodSum = 40.7

**2.6 Attributes Final and Static.****Final****Final Variable:**

- . The value of a variable declared final cannot be changed in the program. It makes the variable a constant. A few examples of declarations are as follows:
  - final double PI = 3.14159; // The value of PI cannot be changed in its scope
  - final int M = 900; // The value of M cannot be changed in its scope
  - final double X = 7.5643; // The value of x cannot be changed in its scope.

- . As mentioned in the comments, the values of PI, M, and x cannot be changed in their respective scopes.

**Final Method:**

- . The attribute final may be used for methods as well as for classes. These are basically connected with inheritance of classes.
  - o When final keyword is used with Java method, it becomes the final method.
  - o A final method cannot be overridden in a sub-class.

**Final Class:**

- . A Java class with final modifier is called final class. A final class cannot be subclassed or inherited. Several classes in Java are final including String, Integer, and other wrapper classes.
- . There are certain important points to be noted when using final keyword in Java
  - a. New value cannot be reassigned to a variable defined as final in Java.
  - b. Final keyword can be applied to a member variable, local variable, method, or class.
  - c. Final member variable must be initialized at the time of declaration.
  - d. Final method cannot be overridden in Java
  - e. Final class cannot be inheritable in Java
  - f. Final is different from finally keyword, which is used on Exception handling in Java

**Example- 1:** Java Program to demonstrate the final keyword.

```
public class Final
{
 public static void main (String args[])
 {
 int n =10; // Normal variable
 final int f= 20; // final variable
 System.out.println("n = "+ n);
 System.out.println("f = "+ f);
 n = 50; // Now the value 50 is assigned to variable n
 f= 60; // Error : f is final variable can not be changed
 }
}
```

**Output:**

C:\>javac Final.java

Final.java:13: error: cannot assign a value to final variable f

f= 60; // Error : f is final variable can not be changed

^

1 error

C:\>

**Example- 1:** Java Program to demonstrate the final keyword.

```
public class Final
```

```
{
```

```
 public static void main (String args[])
```

```
 {
```

```
 int n =10; // Normal variable
```

```
 final int f= 20; // final variable
```

```
 System.out.println("n = "+ n);
```

```
 System.out.println("f = "+ f);
```

```
 n = 50; // Now the value 50 is assigned to variable n
```

```
 System.out.println("n = "+ n);
```

```
 }
```

```
}
```

**Output:**

C:\>javac Final.java

C:\>java Final

n = 10

f = 20

n = 50

## Static

### Static Variables:

- . The static variables are class variables. Only one copy of such variables is kept in the memory and all the objects share that copy.
- . The static variables are accessed through class reference, whereas the instance variables are accessed through class object reference.
- . The variables in a class may be modified by modifier static.
- . The non-static variables declared in a class are instance variables Each object of the class keeps a copy of the values of these variables.

**Static Methods:**

- . The static methods are similar to class methods and can be invoked without any reference of object of class, however, class reference (name of class) is needed, as in the following example The method like **sqrt()** is declared as static method in **Math** class and is called

Math.sqrt(5); // Finds square root of 5

- . The static method is called using the method name that is preceded by the class name; in this case. Math and period ().

**Example :** Java program to illustration of using static methods of class Math

```
public class StaticMethods
```

```
{
```

```
 public static void main (String args[])
```

```
 {
```

```
 System.out.println("The Square root root of 16 = "+ Math.sqrt(16));
```

```
 System.out.println("The cubroot root of 27 = "+ Math.cbrt(27));
```

```
 //printing five random variables
```

```
 for(int i = 1; i<=5;i++)
```

```
 System.out.println("Random Number"+i+ " = " + (int)(100 *Math.random()));
```

```
 }
```

```
}
```

**Output:**

```
E:\>javac StaticMethods.java
```

```
E:\>java StaticMethods
```

The Square root root of 16 = 4.0

The cubroot root of 27 = 3.0

Random Number 1 = 77

Random Number 2 = 69

Random Number 3 = 83

Random Number 4 = 2

Random Number 5 = 66

E:\>java StaticMethods

The Square root root of 16 = 4.0

The cubroot root of 27 = 3.0

Random Number 1 = 67

Random Number 2 = 31

Random Number 3 = 10

Random Number 4 = 13

Random Number 5 = 40

### **Additional Resources – ALL NPTEL LINKS/OTHER TEXT BOOKS**

- 1) <https://nptel.ac.in/courses/106/105/106105191/>
- 2) <https://www.coursera.org/specializations/object-oriented-programming>
- 3) <https://www.edx.org/course/java-programming-fundamentals?index=product&queryID=6fa5777a91ae2b01cd3243daf1d57b74&position=7>
- 4) [https://infyspringboard.onwingspan.com/web/en/app/toc/lex\\_auth\\_012880464547618816347\\_shared/overview](https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_012880464547618816347_shared/overview)
- 5) <https://www.w3schools.com/java/default.asp>
- 6) <https://www.javatpoint.com/java-tutorial>
- 7) <https://www.geeksforgeeks.org/java/>
- 8) <https://www.tutorialspoint.com/java/index.htm>
- 9) <https://www.programiz.com/java-programming>
- 10) <https://stackify.com/java-tutorials/>

**Assignment cum Tutorial questions****Part-A****Objective Questions (1M)****MODULE 1: Classes and Objects**

- 1) Which of the following is the correct syntax for creating Object [       ]  
A) Classname objName=new Classname;                      B)Classname objName=new Classname();  
C)Classname objName=Classname();                      D)objName classname=new objName();
- 2) \_\_\_\_\_ is a keyword that refers to the current object that invoked the method.
- 3) \_\_\_\_\_ is the process of reclaiming the runtime unused memory automatically.
- 4) What is the output of this program? [       ]

```
class box
{
 int width;
 int height;
 int length;
 int vol;
 box()
 {
 width = 5;
 height = 5;
 length = 6;
 }
 void volume()
 {
 vol = width*height*length;
 }
}

class constructor_output
{
 public static void main(String args[])
 {
 box obj = new box();
 obj.volume();
 System.out.println(obj.vol);
 }
}
```

$$\}$$
$$\}$$

- A) 100                      B) 150                      C) 200                      D) 250

5) Consider the following code

$$[ \quad ]$$

class A

$$\{$$

```
private int i;
```

```
public intj;
```

$$\}$$

class B extends A

$$\{$$

```
int k;
```

```
void show()
```

$$\{$$
$$k=i+j;$$

```
System.out.println("sum of " +i+ "and" +j+"="+k);
```

$$\}$$

```
public static void main(String arg[])
```

 $\{$ 

```
B b1=new B();
```

$$\}$$
$$\}$$

- A) B gets only the member j through inheritance from A

- B) B gets both i, j through inheritance from A

- C) A is the sub class and B is the super class

- D) None of the above

5) The following code prints

$$[ \quad ]$$

class A

$$\{$$

```
int i;
```

```
intj;
```

A0

$$\{$$
$$i = 1;$$



```
j = 2;
}

}

class Output
{
public static void main(String args[])
{
A obj1 = new A();

System.out.print(obj1.toString());
}
}
```

- A. true
- B. false
- C. String associated with object
- D. Compilation Error

6) What is the output of the following Java program?

```
class Point
{
 int m_x, m_y;

 public Point(int x, int y)
 {
 m_x = x; m_y = y;
 }

 public static void main(String args[])
 {
 Point p = new Point();
 }
}
```

- A. 1
- B. Garbage value
- C. Compiler error
- D. runtime error

**MODULE 2: Methods**

- 1) \_\_\_\_\_ is the process of defining 2 or more methods within same class that have same name but different parameter declarations. [       ]

A) Method overriding

B) Method overloading

C) Method hiding

D) None of the above

- 2) Run-time polymorphism is achieved by using \_\_\_\_\_ [       ]

A) Method Overloading

B) Constructor Overloading

C) Method Overriding

D) this keyword

- 3) what is the output of this program? [       ]

```
class overload
```

```
{
 int x;
 int y;
 void add(int a)
 {
 x = a + 1;
 }
 void add(int a, int b)
 {
 x = a + 2;
 }
}
```

```
class Overload_methods
```

```
{
 public static void main(String args[])
 {
 overload obj = new overload();
 int a = 0;
 obj.add(6,7);
 System.out.println(obj.x);
 }
}
```

A) 5

B) 8

C) 7

D) 6

## Part-B

### Short Answer Questions (2M)

#### MODULE 1: Classes and Objects

1. Define class and object. [BL2]
2. Give the syntax of creating object in java. [BL2]
3. Recall and list the parameter passing methods in java. [BL2]
4. Define method. How to pass parameters to methods. [BL2]
5. Identify different types of variables defined in class. [BL2]

#### MODULE 2: Methods

1. Compare constructor method and class methods. [BL2]
2. List out different access modifiers supported in java. [BL2]
3. Difference between static method and final method. [BL2]
4. List out different types of constructors in java. [BL2]

## Part-C

### Descriptive Questions (5M)

#### MODULE 1: Classes and Objects

1. Define a class. Create a class to describe a student object to read and display the details like name, roll number and marks. [BL3]
2. Explain how returning of objects is different from returning the values. [BL2]
3. Illustrate the procedure to define and access method in a class with suitable example program. [BL3]
4. Write a program to print the area of a rectangle by creating a class named 'Area' having two methods. First method named as 'setDim()' takes length and breadth of rectangle as parameters and the second method named as 'getArea()' returns the area of the rectangle. [BL3]
5. What is a Constructor? Explain the different types of constructors and explain with suitable example programs. [BL3]
6. Analyze the outcome of the following java code segment and justify your answer. [BL3]

```
final class A
{
```

```
 int a;
 }
 class B extends A
 {
 void display()
 {
 System.out.println("a="+a);
 }

 public static void main(String arg[])
 {
 B b1=new B();
 b1.display();
 }
 }
```

7. Create a class named 'Student' with no, name, address instance variables, and also write getData() and putData() methods, which are useful to read and display student information (no, name and address) by calling with Student class Object. [BL3]

## MODULE 2: Methods

1. Explain Method Overloading with an example. [BL2]
2. Define constructor. Can we overload a constructor? If so, explain with an example. [BL2]
3. Distinguish between static and final keywords with suitable example. [BL2]
4. Explain different access specifiers supported by Java with an example. [BL2]
5. Write a Java program to overload a method named as area() which computes the area of a geometrical figure based on number of parameters and type. If number of parameters is 1 and is of type float it should calculate the area of circle, if it is of type int it should calculate area of square. If the number of parameters is 2 and they are of type float calculate area of triangle, if they are of int calculate area of rectangle. Outline Method Overriding concept in Java. [BL3]
6. Is it possible to declare two or more methods with the same name within a class? If yes, what are the rules to be followed? Give an example. [BL2]

- To learn how to extend Java classes with inheritance and dynamic binding.

**Syllabus:**

**Arrays:** Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Accessing Elements of Arrays.

**Inheritance:** Introduction, Process of Inheritance, Types of Inheritances, Universal Super Class-Object Class, Inhibiting Inheritance of Class Using Final, Access Control and Inheritance, Multilevel Inheritance, Application of Keyword Super, Constructor Method and Inheritance, Method Overriding, Dynamic Method Dispatch, Abstract Classes, Final class and Methods.

**Interfaces:** Introduction, Declaration of Interface, Implementation of Interface, Multiple Interfaces, Nested Interfaces, Inheritance of Interfaces, Default Methods in Interfaces, Static Methods in Interface, Functional Interfaces, Annotations.

**Learning Outcomes:**

Students will be able to

utilize arrays, implement various inheritance techniques, and work with interfaces in Java to create object-oriented programs.

- make use of arrays in java application development.
- apply inheritance to a java application.
- demonstrate the interface and its features using java application.
- implement multiple inheritance using interface.

**UNIT-3****Arrays, Inheritance and Interfaces****Modules:**

- 1) Arrays.
- 2) Inheritance.
- 3) Interfaces.

**Objective:**

- To learn how to extend Java classes with inheritance and dynamic binding.

**Syllabus:**

**Arrays:** Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Accessing Elements of Arrays.

**Inheritance:** Introduction, Process of Inheritance, Types of Inheritances, Multilevel Inheritance, Application of Keyword Super, Method Overriding, Dynamic Method Dispatch, Abstract Classes, Final class and Methods.

**Interfaces:** Introduction, Declaration of Interface, Implementation of Interface, Multiple Interfaces, Nested Interfaces.

**Module 1: Arrays****1.1 Introduction**

An array is a collection of elements that are accessed by index. The index is an integer value that indicates the position of an element within the array. In Java, arrays are zero-indexed, meaning the first element is at index 0, the second element is at index 1, and so on.

**Key Characteristics of Arrays:**

- **Fixed Size:** Once an array is created, its size cannot be changed. The number of elements it can hold is fixed.
- **Homogeneous Elements:** All elements in an array must be of the same type, whether it's integers, doubles, or objects.
- **Indexed Access:** Elements are accessed via their index, which allows for quick retrieval and manipulation.

## 1.2 Declaring and Initialization of Arrays:

**Declaration:** You declare an array by specifying the type of elements it will hold, followed by square brackets [].

```
int[] numbers; // Declares an array of integers
```

```
String[] names; // Declares an array of strings
```

**Instantiation:** You create an array instance using the new keyword, specifying the number of elements.

```
numbers = new int[5]; // Creates an array of 5 integers
```

```
names = new String[10]; // Creates an array of 10 strings
```

**Initialization:** You can also initialize an array at the time of declaration.

```
int[] ages = {25, 30, 35}; // Initializes an array with 3 integers
```

```
String[] colors = {"Red", "Green", "Blue"}; // Initializes an array with 3 strings
```

## 1.3 Storage of Array in Computer Memory

The memory allocation of arrays in Java is a well-defined process that involves several key aspects:

### 1.3.1 Array Declaration and Initialization

**Declaration:** Arrays in Java are declared using a specific type and a pair of square brackets. *For example:* `int[] arr;` declares an array of integers. At this stage, the array does not yet occupy memory as it only specifies the type of elements it will hold.

**Initialization:** To allocate memory for the array, use the new keyword followed by the type and the size of the array. *For example,* `arr = new int[10];` creates an array capable of holding 10 integers. This operation allocates a contiguous block of memory large enough to store 10 int elements.

### 1.3.2 Contiguous Memory Allocation

Arrays are allocated in contiguous memory locations, the entire array is stored in a single, continuous block of memory. The first element is stored at a base memory address, and subsequent elements are stored at sequential addresses. This layout facilitates efficient indexing, as accessing any element can be done directly by calculating its memory address based on the base address and the size of each element.

### 1.3.3. Array Object Structure

**Array Reference:** The variable that holds the array is a reference to the array object. This reference itself is typically stored on the stack for local variables or in the heap for objects, depending on the context.

**Array Object:** The array object is stored in the heap memory and includes:

**Header Information:** This metadata contains information about the array such as its length and type. The length is crucial as it helps ensure that index bounds are checked during runtime.

**Elements:** The actual data elements of the array are stored in contiguous locations after the header.

### 1.3.4. Zero-Based Indexing

Java arrays use zero-based indexing. This means the first element is accessed with index 0, the second element with index 1, and so forth. This indexing scheme simplifies the calculation of memory addresses, as the offset of an element can be directly computed by multiplying the index by the size of the element.

### 1.3.5. Fixed Size

Once an array is created, its size is fixed and cannot be changed. If a different size is needed, a new array must be created, and the elements from the old array must be copied to the new one. This fixed-size nature is crucial for maintaining efficient access but requires additional handling for dynamic sizing needs.

### 1.3.6. Memory Layout Example

```
int[] numbers = new int[5];
```



```
numbers[0] = 10;
```

```
numbers[1] = 20;
```

```
numbers[2] = 30;
```

```
numbers[3] = 40;
```

```
numbers[4] = 50;
```

**Declaration:** `int[] numbers;` - Creates a reference to an integer array.

**Initialization:** `numbers = new int[5];` - Allocates memory for 5 integers in a contiguous block.

### Memory Layout:

The array object resides in the heap with a header that contains metadata.

The elements 10, 20, 30, 40, 50 are stored in contiguous memory locations, each occupying space according to the int data type.

### 1.4 Accessing Elements of Array

You access elements using their index. For example:

```
int firstAge = ages[0]; // Gets the first element (25) from the ages array
```

```
String favoriteColor = colors[2]; // Gets the third element ("Blue") from the colors array
```

### Common Operations:

Iteration: You often use loops to process arrays.

```
for (int i = 0; i < numbers.length; i++) {

 System.out.println(numbers[i]); // Prints each element in the numbers array

}
```

**Length:** You can get the number of elements in an array using the length property.

```
int arraySize = numbers.length; // Gets the size of the numbers array
```

**Multi-dimensional Arrays:** Java supports arrays of arrays, allowing you to create matrices or tables.

```
int[][] matrix = new int[3][3]; // Creates a 3x3 two-dimensional array
```

```
matrix[0][0] = 1; // Sets the value of the element at row 0, column 0
```

Arrays are a powerful tool in Java, providing a way to manage and manipulate collections of data efficiently.

## Module 2: Inheritance

### 2.1 Introduction

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows one class (the child or subclass) to inherit properties and behaviors from another class (the parent or superclass). This mechanism promotes code reusability and establishes a natural hierarchy between classes. Here's a brief introduction:

#### Key Concepts of Inheritance

1. Superclass (Parent Class):

- The superclass is the class that provides the common properties and methods to be inherited by other classes. It defines the base functionality that other classes can build upon.

2. Subclass (Child Class):

- The subclass inherits attributes and methods from the superclass. It can also have additional attributes and methods or override those inherited from the superclass.

3. Reusability:

- Inheritance promotes code reusability by allowing the subclass to reuse the code defined in the superclass. This reduces redundancy and simplifies maintenance.

4. Hierarchy:

- Inheritance establishes a hierarchical relationship between classes. For example, if Bird is a superclass, then Sparrow and Eagle can be subclasses of Bird.

#### Basic Syntax in Java

In Java, inheritance is implemented using the **extends** keyword.

**Superclass Example:**

```
public class Animal {

 String name;

 public void eat() {

 System.out.println(name + " is eating.");

 }

}

public class Dog extends Animal {

 public void bark() {

 System.out.println(name + " is barking.");

 }

}

public class Main {

 public static void main(String[] args) {

 Dog myDog = new Dog();

 myDog.name = "Buddy";

 myDog.eat(); // Inherited method

 myDog.bark(); // Method defined in subclass

 }

}
```

## 2.2 Process of Inheritance

Inheritance is a core principle in object-oriented programming (OOP) that allows a new class to inherit the properties and behaviors of an existing class. The process of inheritance typically involves several steps, from defining the superclass to using the subclass in a program. Here's a step-by-step overview of the inheritance process:

### 1. Define the Superclass

The superclass (or parent class) is the base class that contains common attributes and methods that you want to share with other classes.

#### Example:

```
public class Animal {

 String name;

 public void eat() {

 System.out.println(name + " is eating.");

 }

 public void sleep() {

 System.out.println(name + " is sleeping.");

 }

}
```

### 2. Create the Subclass

The subclass (or child class) extends the superclass using the extends keyword. This allows the subclass to inherit the properties and methods of the superclass. The subclass can also define additional properties and methods or override existing ones.

**Example:**

```
public class Dog extends Animal {

 public void bark() {

 System.out.println(name + " is barking.");

 }

 // Overriding the eat method

 public void eat() {

 System.out.println(name + " is eating dog food.");

 }

}
```

**3. Use the Subclass**

Instantiate the subclass and use it in your program. The subclass has access to the inherited members from the superclass, as well as any additional members defined in the subclass.

**Example:**

```
public class Main {

 public static void main(String[] args) {

 // Create an instance of the subclass

 Dog myDog = new Dog();// Set

 properties inherited from the

 superclass myDog.name = "Buddy";

 // Call inherited methods

 myDog.eat(); // Calls the overridden method in Dog
```

```
 myDog.sleep(); // Inherited from Animal

 // Call subclass-specific method

 myDog.bark();

}

}
```

### **Detailed Steps**

#### **Defining the Superclass:**

Create a class with common attributes (fields) and behaviors (methods) that can be reused by other classes.

#### **Extending the Superclass:**

Create a new class that extends the superclass. This subclass will automatically inherit all accessible members (fields and methods) of the superclass.

#### **Overriding Methods:**

If the subclass needs to provide a specific implementation of a method inherited from the superclass, it can override that method. Use the `@Override` annotation to ensure that you are correctly overriding a method.

#### **Adding New Members:**

The subclass can also add new fields and methods that are specific to it, enhancing its functionality beyond what the superclass provides.

#### **Constructor Initialization:**

Constructors in the subclass are not inherited but can call the superclass constructor using `super()`. This allows initialization of inherited fields.

#### **Using Polymorphism:**

Subclass objects can be referenced by superclass variables, allowing for flexible code where the

specific type of object can be determined at runtime.

Example of Polymorphism:

```
public class Main {

 public static void main(String[] args) {

 Animal myPet = new Dog(); // Polymorphism: Animal reference to a Dog object

 myPet.name = "Rex";

 myPet.eat(); // Calls Dog's overridden eat method

 }

}
```

## 2.3 Types of Inheritance

There are several types of inheritance, each defining a different way in which classes can be related to one another. Here's an overview of the common types of inheritance:

**Single Inheritance:** One class inherits from another class.

**Multiple Inheritance:** A class inherits from multiple classes (often achieved using interfaces in Java).

**Multilevel Inheritance:** A class is derived from another derived class, forming a chain.

**Hierarchical Inheritance:** Multiple classes inherit from a single superclass.

**Hybrid Inheritance:** A mix of different types of inheritance (achieved via interfaces in Java to avoid complexity).

Each type of inheritance helps in organizing and structuring code, allowing for flexible and reusable object-oriented designs.

### 1. Single Inheritance

**Description:**



In single inheritance, a class (subclass) inherits from one and only one superclass (parent class). This is the simplest form of inheritance.

Example:

```
class Animal {

 void eat() {

 System.out.println("Animal eats");

 }

}

class Dog extends Animal {

 void bark() {

 System.out.println("Dog barks");

 }

}
```

## 2. Multiple Inheritance

### Description:

Multiple inheritance allows a class to inherit from more than one class. However, many languages, including Java, do not support multiple inheritance directly for classes to avoid complexity and ambiguity.

Instead, multiple inheritance can be achieved using interfaces. A class can implement multiple interfaces, thereby inheriting behaviors from multiple sources.

### Example:

```
interface Animal {

 void eat();
```

```
}

interface Pet {

 void play();

}

class Dog implements Animal, Pet {

 public void eat() {

 System.out.println("Dog eats");

 }

 public void play() {

 System.out.println("Dog plays");

 }

}
```

### 3. Multilevel Inheritance

#### Description:

In multilevel inheritance, a class is derived from another derived class. This creates a chain of inheritance, where each class inherits from the previous class in the chain.

#### Example:

```
class Animal {

 void eat() {

 System.out.println("Animal eats");

 }

}
```

```
class Mammal extends Animal {

 void breathe() {

 System.out.println("Mammal breathes");

 }
}

class Dog extends Mammal {

 void bark() {

 System.out.println("Dog barks");

 }
}
```

#### 4. Hierarchical Inheritance

##### **Description:**

In hierarchical inheritance, multiple subclasses inherit from a single superclass. This allows different subclasses to share common properties and methods from the same superclass.

##### **Example:**

```
class Animal {

 void eat() {

 System.out.println("Animal eats");

 }
}

class Dog extends Animal {

 void bark() {
```

```
 System.out.println("Dog barks");

 }

}

class Cat extends Animal {

 void meow() {

 System.out.println("Cat meows");

 }

}
```

## 5. Hybrid Inheritance

### Description:

Hybrid inheritance is a combination of two or more types of inheritance. For example, it may combine single, multiple, and hierarchical inheritance. However, some languages like Java avoid hybrid inheritance directly through classes due to complexity and potential issues, but you can achieve similar effects using interfaces.

### Example:

```
interface Animal {

 void eat();

}

interface Pet {

 void play();

}

class Mammal implements Animal {
```

```
public void eat() {

 System.out.println("Mammal eats");

}

}

class Dog extends Mammal implements Pet {

 public void play() {

 System.out.println("Dog plays");

 }

 void bark() {

 System.out.println("Dog barks");

 }

}
```

## 2.4 Multilevel Inheritance

Multilevel inheritance is a type of inheritance in object-oriented programming where a class is derived from another class that is also derived from a third class. This creates a chain of inheritance where each class in the hierarchy inherits attributes and behaviors from its immediate parent class, as well as indirectly from all ancestor classes in the hierarchy.

### Key Concepts

#### Chain of Inheritance:

Multilevel inheritance involves at least three levels: a base class (the grandparent), an intermediate class (the parent), and a derived class (the child). Each class inherits from its immediate predecessor in the chain.

#### Access to Members:

A class at any level inherits all accessible members (fields and methods) from its parent class and indirectly from all ancestor classes up the chain.

**Constructor Initialization:**

When an object of a derived class is created, constructors are called in the order from the top-most ancestor down to the most derived class. This ensures proper initialization of all inherited properties.

**Example****Base Class (Grandparent):**

```
public class Animal {

 void eat() {

 System.out.println("Animal eats");
 }

}
```

**Intermediate Class (Parent):**

```
public class Mammal extends Animal {

 void breathe() {

 System.out.println("Mammal breathes");
 }

}
```

**Derived Class (Child):**

```
public class Dog extends Mammal {

 void bark() {

 System.out.println("Dog barks");
 }

}
```

```
 }
}

public class Main {

 public static void main(String[] args) {

 Dog myDog = new Dog();

 // Access methods from all levels of the hierarchy

 myDog.eat(); // From Animal

 myDog.breathe(); // From Mammal

 myDog.bark(); // From Dog

 }
}
```

### **Key Points:**

#### **Hierarchy:**

In multilevel inheritance, each derived class inherits from its immediate parent class and indirectly from all ancestor classes in the chain.

#### **Method Overriding:**

Methods can be overridden in any class in the hierarchy. The most derived class's implementation will be used if it overrides a method from any ancestor class.

#### **Constructor Execution:**

Constructors are executed in the order from the top-most class down to the most derived class, ensuring that all parent class constructors are called before the derived class constructor.

#### **Polymorphism:**

Objects of the most derived class can be referred to using references of any of their ancestor classes, allowing for flexible code and method invocation.

Multilevel inheritance allows you to create a hierarchy of classes where each class inherits from a previous class in the chain. This type of inheritance supports code reuse and hierarchical organization, modeling relationships that reflect real-world structures. It requires careful management to avoid complexities, but it is a powerful tool for creating organized and modular code.

## 2.5 Application of keyword Super

In Java, the super keyword is used within a subclass to refer to its superclass. It is a fundamental part of inheritance and serves several important purposes:

### Accessing Superclass Methods:

super allows a subclass to call methods defined in its superclass. This is useful when you want to use or extend the functionality of a superclass method while still keeping its original behavior.

### Example:

```
class Animal {

 void eat() {

 System.out.println("Animal eats");

 }

}

class Dog extends Animal {

 void eat() {

 super.eat(); // Calls the eat() method from Animal

 System.out.println("Dog eats dog food");

 }

}
```



```
}

public class Main {

 public static void main(String[] args) {

 Dog myDog = new Dog();

 myDog.eat();

 }

}
```

**Output:**

Animal eats

Dog eats dog food

**Accessing Super class Fields:**

super can be used to access fields of the superclass when there is a name conflict with fields in the subclass. This is done using super.fieldName.

**Example:**

```
class Parent {

 String name = "Parent";

 void showName() {

 System.out.println("Parent name: " + name);

 }

}

class Child extends Parent {

 String name = "Child";
```

```
void showName() {

 System.out.println("Child name: " + name);

 System.out.println("Parent name: " + super.name); // Accesses Parent's name

}

}

public class Main {

 public static void main(String[] args) {
 Child c = new Child();

 c.showName();

 }

}
```

**Output:**

Child name: Child

Parent name: Parent

**Calling Superclass Constructor:**

super is used to call a constructor of the superclass. This is especially useful when you need to initialize the superclass part of an object before initializing the subclass-specific part.

Example:

```
class Animal {

 Animal(String name) {

 System.out.println("Animal constructor with name: " + name);

```

```
}

}

class Dog extends Animal {

 Dog(String name) {

 super(name); // Calls Animal constructor

 System.out.println("Dog constructor");

 }
}

public class Main {

 public static void main(String[] args) {

 Dog myDog = new Dog("Buddy");

 }
}
```

**Output:**

Animal constructor with name: Buddy

Dog constructor

**The super keyword in Java is used to:**

- Access Super class Methods: Call methods defined in the super class that may be overridden in the subclass.
- Access Super class Fields: Access fields of the super class, especially when there is a name conflict with fields in the subclass.
- Call Super class Constructors: Initialize the super class part of an object when constructing an instance of a subclass.

By using super, you can ensure that your subclass properly utilizes and extends the functionality provided by its super class.

## 2.6 Inheritance and Method Overriding:

- In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to *override* the method in the superclass.
- When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass. The version of *the method defined by the superclass will be hidden*.

Example: Java Program to demonstrate overriding (Run-time Polymorphism).

```
import java.io.*;

class A

{

 public int i;

 public A()

 {

 System.out.println("\n \t default constructor A() is called");

 i=10;

 }

 public void display()

 {

 System.out.println("\n \t in A class i= "+i);

 }

}
```

```
}
```

```
class B extends A
```

```
{
```

```
 public int j;
```

```
 public B()
```

```
 {
```

```
 System.out.println("\n \t default constructor B() is called");
```

```
 j=20;
```

```
 }
```

```
 public void display()
```

```
 {
```

```
 j=i+1;
```

```
 System.out.println("\n \t in B class j= "+j);
```

```
 }
```

```
}
```

```
public class OverRiding
```

```
{
```

```
 public static void main(String ar[])throws IOException
```

```
 {
```

```
 System.out.println("\n \t start of main()");
```

```
 B b=new B();
```

```
 b.display(); //display() in B class override the display() in A class
```

```
 System.out.println("\n \t end of main()");

 }

}
```

## OUT PUT

start of main()

default constructor A() is called

default constructor B() is called

in B class j= 21

end of main

## 2.6 Method Overriding

Method overriding is a key concept in object-oriented programming, particularly in Java. It allows a subclass to provide a specific implementation for a method that is already defined in its super class. This concept enables polymorphism, allowing objects to be treated as instances of their super class while executing subclass-specific behaviors.

### Key Concepts of Method Overriding

#### Purpose:

To modify or extend the behavior of a method inherited from a super class.

Allows a subclass to provide a more specific implementation of a method that is already defined in its parent class.

#### Method Signature:

The overridden method in the subclass must have the same name, return type, and parameter list as the method in the super class. The method signature must be identical.

**Access Modifiers:**

The overriding method in the subclass can have the same or a less restrictive access modifier compared to the method in the super class.

**For example,** if the super class method is protected, the overriding method can be protected or public, but not private.

Method overriding is used to achieve runtime polymorphism, where the method that gets executed is determined at runtime based on the object's type.

**Example of Method Overriding**

```
class Animal {

 void makeSound() {

 System.out.println("Animal makes a sound");

 }

}

class Dog extends Animal {

 void makeSound() {

 System.out.println("Dog barks");

 }

}

public class Main {

 public static void main(String[] args) {

 Animal myAnimal = new Dog(); // Upcasting

 myAnimal.makeSound(); // Calls Dog's makeSound() method

 }
}
```

```
}
```

**Output:**

Dog barks

**Rules for Method Overriding****Method Signature:**

The method in the subclass must have the same name, return type, and parameter list as the method in the superclass.

**Access Level:**

The access level of the overriding method cannot be more restrictive than the method in the super class. For example, if the super class method is protected, the overriding method can be protected or public, but not private.

**Return Type:**

The return type of the overriding method must be the same as or a subtype of the return type of the method in the super class (covariant return types).

**Exceptions:**

The overriding method cannot throw new or broader checked exceptions than those declared in the method of the super class. However, it can throw fewer or more specific exceptions.

**Static Methods:**

Static methods cannot be overridden. If a static method is defined with the same name and parameters in the subclass, it is considered method hiding, not overriding.

**Example with Exception Handling**

```
class Animal {

 void eat() throws IOException {
```



```
 System.out.println("Animal eats");
 }
}

class Dog extends Animal {

 void eat() throws FileNotFoundException { // Can throw a more specific exception

 System.out.println("Dog eats dog food");

 }

}
```

### **Example with Covariant Return Types**

```
class Animal {

 Animal getAnimal() {

 return this;

 }

}

class Dog extends Animal {

 Dog getAnimal() { // Covariant return type

 return this;

 }

}
```

- Method overriding allows a subclass to provide a specific implementation for a method inherited from its super class.

- The overriding method must have the same name, return type, and parameter list as the method in the super class.
- It enables runtime polymorphism and is essential for dynamic method dispatch.

## 2.7 Dynamic Method Dispatch

Dynamic method dispatch is a fundamental concept in object-oriented programming, particularly in Java. It allows the Java runtime to determine at runtime which method implementation to invoke when a method is called on an object. This concept is closely tied to polymorphism and method overriding.

### Key Concepts of Dynamic Method Dispatch

#### Polymorphism:

Dynamic method dispatch is a form of polymorphism, which means that a single method call can execute different code based on the type of object that invokes it.

#### Method Overriding:

For dynamic method dispatch to work, a subclass must override a method defined in its super class. The overridden method in the subclass provides the specific implementation for the method call.

#### Method Resolution at Runtime:

During runtime, Java determines the actual class of the object and calls the appropriate overridden method based on that class. This means that the method that gets executed is determined by the object's runtime type, not the reference type.

### How Dynamic Method Dispatch Works

#### Method Invocation:

When a method is called on an object, Java looks up the method in the object's class. If the method is overridden, the version in the most derived class is executed.

### Example of Dynamic Method Dispatch

```
class Animal {

 void makeSound() { System.out.println("Animal
 makes a sound");

 }
}

class Dog extends Animal { void

 makeSound() {

 System.out.println("Dog barks");

 }
}

class Cat extends Animal { void

 makeSound() {

 System.out.println("Cat meows");

 }
}

public class Main {

 public static void main(String[] args) { Animal

 myAnimal;

 myAnimal = new Dog(); // Upcasting myAnimal.makeSound(); //

 Calls Dog's makeSound() method
```

```
myAnimal = new Cat(); // Upcasting myAnimal.makeSound(); //
```

Calls Cat's makeSound() method

```
}
```

```
}
```

**Output:**

Dog barks Cat

meows

**Benefits of Dynamic Method Dispatch**

**Flexibility:** Allows for methods to be overridden in subclasses, providing specific behavior that can be invoked at runtime.

**Extensibility:** New subclasses can be added without modifying existing code, enhancing maintainability and scalability.

**Polymorphism:** Enables writing more generic code that can handle objects of various subclasses in a uniform manner.

Dynamic method dispatch is a mechanism in Java that enables runtime method resolution. It allows a subclass to provide a specific implementation of a method that is already defined in its super class. At runtime, the Java Virtual Machine (JVM) determines the appropriate method to execute based on the actual class of the object, not the reference type. This provides flexibility and supports polymorphism, making Java programs more modular and extensible.

**2.8 Abstract class**

Abstract classes are a fundamental concept in object-oriented programming. They serve as a blueprint for other classes. An abstract class cannot be instantiated directly; instead, it defines a common interface for other classes. Here are some key points:

1. Purpose: Abstract classes provide a base for creating other classes. They can include abstract methods (methods without implementation) that must be implemented by subclasses, as well as concrete methods (with implementation).
2. Abstract Methods: These are methods declared in the abstract class that do not have a body. Subclasses that inherit from the abstract class must provide concrete implementations of these methods.
3. Concrete Methods: Abstract classes can also have fully implemented methods that can be used or overridden by subclasses.
4. Inheritance: Subclasses of an abstract class inherit its abstract methods and are required to implement them, providing specific behaviors while still adhering to a common interface.
5. Usage: Abstract classes are useful for defining a common base and enforcing a certain structure in your code, while allowing for flexibility in the implementation details provided by subclasses.

### **Abstract Class Example in Java**

```
// Abstract class

abstract class

Animal {

 // Abstract method (does not have a

 body) abstract void makeSound();

 // Concrete

 method void

 sleep() {

 System.out.println("Zzz...");
```

```
 }

}

// Concrete class inheriting from Animal
class Dog extends Animal {

 // Implementation of the abstract method

 @Override

 void makeSound() {

 System.out.println("Woof!");

 }

}

// Another concrete class inheriting from

Animal class Cat extends Animal {

 // Implementation of the abstract

 method void makeSound() {

 System.out.println("Meow!");

 }

}

// Main class to test the abstract class and its subclasses

public class Main {

 public static void main(String[] args) {

 Animal[] animals = { new Dog(), new

 Cat() }; for (Animal animal : animals) {
```

```
 animal.makeSound(); // Outputs: Woof!

 Meow! animal.sleep(); // Outputs: zz...

 }

}

}
```

- Animal is an abstract class with one abstract method makeSound() and one concrete method sleep().
- Dog and Cat are concrete subclasses that provide specific implementations of the makeSound() method.
- In the Main class, we create an array of Animal that holds instances of Dog and Cat. When iterating through this array, each animal makes its own specific sound and then sleeps, demonstrating the use of both abstract and concrete methods.

## 2.9 Final Class and Methods

### Final Class:

- A class that cannot be further extended is called a final class in Java.
- A final class is declared using the final keyword.
- There cannot be any subclass inheritances of a final class, as the compiler will display error notes on trying to inherit a final class during a compilation.

Example: Java Program to demonstrate final class.

```
final class A

{

 int a;

 A(int x)
```

```
{

a=x;
}

void display()

{

System.out.println("a = "+ a);

}

}

class B extends A

{

int b;

B(int x,int y)

{

super(x);

this.b=y;

}

void display()

{

System.out.println("b = "+ b);

}

}

class FinalClass{
```



```
public static void main (String args[])

 {

 A objA= new A(10);

 B objB= new B(100,200);

 objA.display();

 objB.display();

 }

}
```

Output:

C:\>javac FinalClass.java

FinalClass.java:11: error: cannot inherit from final A

class B extends A

^

1 error

### Final Methods:

You can also use the final keyword to prevent methods from being overridden by subclasses. This is useful when you want to ensure that certain methods maintain their original implementation in subclasses.

### Example:

```
public class BaseClass {

 public final void finalMethod() {

 System.out.println("This method cannot be overridden.");
```

```
}

}

public class SubClass extends BaseClass {

 // This will cause a compilation error

 // public void finalMethod() {

 // System.out.println("Attempt to override.");

 // }

}
```

### Final Variables:

When applied to variables, the final keyword indicates that the variable's value cannot be changed once assigned. This is different from preventing inheritance but is another use of final in Java.

## Module 3: Interfaces

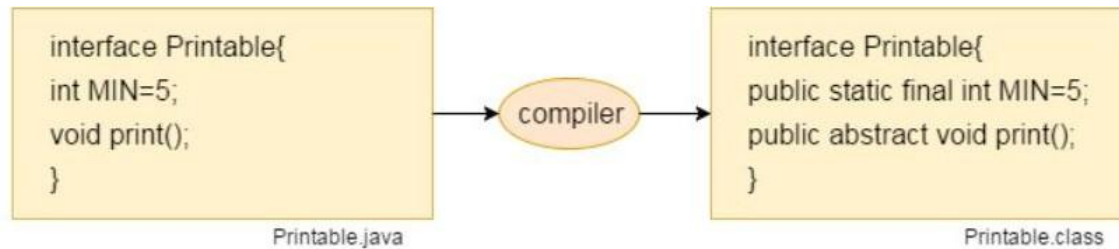
### 3.1 Introduction

**Definition:** An interface is a reference type in Java (and many other languages) that can contain only constants, method signatures, default methods, static methods, and nested types. Interfaces cannot contain instance fields. Classes implement interfaces, and interfaces are implemented by classes.

in Java it is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

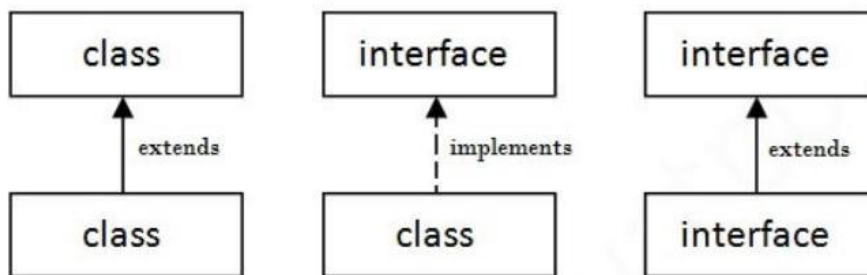
Java Interface also represents the IS-A relationship.

Interface fields are public, static and final by default, and the methods are public and abstract.



### The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a class implements an interface.



### Key Points:

1. Purpose: Define a contract that classes can implement. Interfaces allow different classes to promise that they will provide implementations for the methods defined in the interface.
2. Method Signatures: Interfaces can include abstract methods (methods without a body) that must be implemented by any class that implements the interface.
3. Multiple Inheritance: Interfaces support multiple inheritance. A class can implement multiple interfaces, which allows a class to adhere to multiple contracts.
4. Default Methods: In Java 8 and later, interfaces can have default methods with a body, providing a default implementation. This helps in adding new methods to interfaces without breaking existing implementations.

### Uses of Interfaces in Java

- It is used to achieve total abstraction.

- Since java does not support multiple inheritances in the case of class, by using an interface it can achieve multiple inheritances.
- Any class can extend only 1 class, but can any class implement an infinite number of interfaces.
- It is also used to achieve loose coupling.
- Interfaces are used to implement abstraction.

### Comparison:

- **Inheritance:**

- Provides a mechanism to reuse code by inheriting from a super class.
- A class can inherit from only one super class (single inheritance).

- **Interfaces:**

- Provides a way to define a contract that can be implemented by multiple classes.
- Allows a class to implement multiple interfaces, overcoming the limitation of single inheritance.

inheritance is used for creating a hierarchical relationship and code reuse, while interfaces are used for defining a contract that multiple classes can implement, facilitating flexibility and multiple inheritance.

## 3.2 Declaration of Interface

In programming, an interface is a contract that defines a set of methods or properties that a class must implement, but does not provide the actual implementation for those methods. It essentially specifies what a class should do, but not how it should do it.

### Syntax:

```
interface <interface_name>
```

```
{

 // declare constant fields

 // declare methods that abstract

 // by default.

}
```

**Example:**

```
public interface MyInterface {

 void method1();

 int method2(String input);

}
```

**MyInterface** is the name of the interface. **method1** and **method2** are abstract methods that any class implementing this interface must provide concrete implementations for.

### 3.3 Implementation of interface

To implement an interface, use the keyword implements

```
interface Bank{

 float rateOfInterest();

}

class SBI implements Bank{

 public float rateOfInterest(){return 9.15f;}

}
```

```
class PNB implements Bank{

public float rateOfInterest(){return 9.7f;}

}

class TestInterface2{

public static void main(String[] args){

Bank b=new SBI();

System.out.println("ROI: "+b.rateOfInterest());

}

}
```

### 3.4 Multiple Interfaces

Java supports multiple interface implementation without any issues. A class can implement more than one interface.

#### **Example:**

```
// Define multiple interfaces

public interface InterfaceA {

 void method();

}

public interface InterfaceB {

 void method();

}

// Implement multiple interfaces
```

```
public class MyClass implements InterfaceA, InterfaceB {

 public void method() {

 System.out.println("methodA implemented");

 }

}
```

### 3.5 Nested interfaces

An interface can be nested within another interface or class. Nested interfaces are often used to group related interfaces together and can be either public or package-private (default visibility).

**Example:**

```
public class OuterClass {

 // Nested interface within a class

 public interface NestedInterface {

 void nestedMethod();

 }

 // Another nested interface

 public interface AnotherNestedInterface {

 void anotherNestedMethod();

 }

}
```

// Implementing nested interfaces

```
public class MyClass implements OuterClass.NestedInterface, OuterClass.AnotherNestedInterface
{
 public void nestedMethod() {
 System.out.println("nestedMethod implemented");
 }

 public void anotherNestedMethod() {
 System.out.println("anotherNestedMethod implemented");
 }
}
```



**Assignment cum Tutorial questions****Part-A****Objective Questions (1M)****Module 1: Arrays**

1. Which of these operators is used to allocate memory to array variable in Java? [      ]  
a) malloc                                      b) alloc  
c) new                                              d) new malloc
2. How do you declare an array in Java? [      ]  
a) `int[] arr;`                                      b) `int arr[];`  
c) `int arr;`                                              d) `Array<int> arr;`
3. How do you access an element in an array in Java? [      ]  
a) By using the element's value                                      b) By using the element's index  
c) By using the element's key                                              d) By using the element's label
4. Which method is used to copy one array into another in Java? [      ]  
a) `copy()`                                              b) `clone()`  
c) `System.arraycopy()`                                              d) `Arrays.copy()`
5. What happens if you try to access an array element with an index that is out of bounds? [      ]  
a) It throws an `ArrayIndexOutOfBoundsException`.  
b) The program terminates abruptly  
c) The element value is set to null  
d) The element value is set to 0
6. How do you determine the number of elements in an array? [      ]  
`int buses[] = new int[5];`  
a) `buses.length`                                              b) `buses.length()`  
c) `buses.size`                                              d) `buses.size()`
7. Which of the following creates an empty two-dimensional array with dimensions 2×2? [      ]  
a) `int[][] blue = new int[2, 2];`                                      b) `int[][] blue = new int[2], [2];`  
c) `int[][] blue = new int[2][2];`                                              d) `int[][] blue = new int[2 x 2];`

8. How many of the following are legal declarations? [      ]

String lion [] = new String[] {"lion"};

String tiger [] = new String[1] {"tiger"};

String bear [] = new String[] {};

String cat [] = new String[0] {};

- a) None                      b) One  
c) Two                      d) Three

9. How many of the following are legal declarations? [      ]

float[] lion = new float[];

float[] tiger = new float[1];

float[] bear = new[] float;

float[] cat = new[1] float;

- a) None              b) One              c) Two              d) Three

10. Which is not a true statement about an array? [      ]

- a) An array expands automatically when it is full.  
b) An array is allowed to contain duplicate values.  
c) An array understands the concept of ordered elements.  
d) An array uses a zero index to reference the first element.

11. What is a possible output of the following code? [      ]

String[] strings = new String[2];

System.out.println(strings);

- a) [null, null]                      b) [,]  
c) [Ljava.lang.String;@74a14482                      d) None of the above

12. What does the following output? [      ]

String[] os = new String[] { "Mac", "Linux", "Windows" };

Arrays.sort(os);

System.out.println(Arrays.binarySearch(os, "Mac"));

- a) 0                      b) 1                      c) 2                      d) The output is not defined.

13. Which of these is an incorrect Statement? [      ]

- a) It is necessary to use new operator to initialize an array
- b) Array can be initialized using comma separated expressions surrounded by curly braces
- c) Array can be initialized when they are declared
- d) None of the mentioned

14. Which of these is necessary to specify at time of array initialization? [      ]

- a) Row
- b) Column
- c) Both Row and Column
- d) None of the mentioned

15. What will be the output of the following Java code? [      ]

```
class array_output
{
 public static void main(String args[])
 {
 int array_variable [] = new int[10];
 for (int i = 0; i < 10; ++i)
 {
 array_variable[i] = i;
 System.out.print(array_variable[i] + "");
 i++;
 }
 }
}
```

- a) 0 2 4 6 8
- b) 1 3 5 7 9
- c) 0 1 2 3 4 5 6 7 8 9
- d) 1 2 3 4 5 6 7 8 9 10

**Module 2: Inheritance**

1. What is Inheritance in Java [      ]
  - a) The process of acquiring properties and behaviors of one class by another
  - b) The process of creating objects
  - c) The process of encapsulation
  - d) The process of overloading methods
2. Which of this keyword must be used to inherit a class? [      ]
  - a) super                      b) this
  - c) extent                      d) extends
3. Which of these is correct way of inheriting class A by class B? [      ]
  - a) class B + class A {}                      b) class B inherits class A {}
  - c) class B extends A {}                      d) class B extends class A {}
4. What is a super class in Java? [      ]
  - a) The class that inherits properties and behaviors
  - b) The child class
  - c) The class that is inherited from
  - d) The class that is marked as "final"
5. Can we perform multiple inheritance in Java? [      ]
  - a) Yes                              b) No
6. Which two classes use the Shape class correctly? [      ]
  - a)      public class Circle implements Shape  
        {  
            private int radius;  
        }
  - b)      public abstract class Circle extends Shape  
        {  
            private int radius;  
        }
  - c)      public class Circle extends Shape  
        {

```
private int radius;
public void draw();
}
```

d) public abstract class Circle implements Shape

```
{
 private int radius;
 public void draw();
}
```

e) public class Circle extends Shape

```
{
 private int radius;
 public void draw()
 {
 /* code here */
 }
}
```

f) public abstract class Circle implements Shape

```
{
 private int radius;
 public void draw()
 {
 /* code here */
 }
}
```

a) B,E

b) A,C

c) C,E

d) T,H

7. What will be the output of the following Java program?

[      ]

```
class A
{
 int i;
 void display()
 {
```

```
 System.out.println(i);
 }
}
class B extends A
{
 int j;
 void display()
 {
 System.out.println(j);
 }
}
class inheritance_demo
{
 public static void main(String args[])
 {
 B obj = new B();
 obj.i=1;
 obj.j=2;
 obj.display();
 }
}
```

- a) 0                      b) 1                      c) 2                      d) Compilation Error

8. What is the primary use of the super keyword in Java? [       ]

- a) To create an instance of the superclass.    b) To call a method defined in the superclass.  
c) To access private members of the superclass.    d) To define a method in the subclass.

9. Which of the following statements about calling a superclass constructor with super() is true?

[       ]

- a) super() must be the first statement in the subclass constructor.  
b) super() can be called from anywhere within the subclass constructor.  
c) super() can be used to call any constructor in the superclass, not just the default one.  
d) super() is used to call the superclass methods but not constructors.

10. How does super work when used with method overriding in a subclass? [      ]

- a) super can be used to override methods in the subclass.
- b) super is used to call the overridden method in the superclass.
- c) super can be used to call both overridden and non-overridden methods in the subclass.
- d) super is not used with method overriding; it is only used with constructors.

11. In the following code, what will super.display() do in the subclass? [      ]

```
class Parent {
 void display() {
 System.out.println("Parent display");
 }
}

class Child extends Parent {
 void display() {
 System.out.println("Child display");
 }
 void show() {
 super.display();
 }
}

public class Test {
 public static void main(String[] args) {
 Child obj = new Child();
 obj.show();
 }
}
```

- a) Print "Child display"
- b) Print "Parent display"
- c) Print "Parent display" followed by "Child display"
- d) Cause a compilation error

12. What is dynamic method dispatch in Java? [      ]

- a) It is a technique to call methods of a superclass from a subclass.
- b) It is a mechanism that allows Java to choose the method to call at runtime based on the object's type.
- c) It is a way to dispatch method calls to different threads.
- d) It is a process that involves explicitly specifying which method to call in the code.

13. Which of the following statements is true about method overriding in the context of dynamic method dispatch? [      ]

- a) The method in the superclass must be static for dynamic method dispatch to work.
- b) The method in the subclass must have the same signature as the method in the superclass.
- c) Dynamic method dispatch is used only when methods are not overridden.
- d) It is possible to use dynamic method dispatch with private methods.

14. Given the following code, what will be the output? [      ]

```
class Animal {
 void makeSound() {
 System.out.println("Animal makes a sound");
 }
}

class Dog extends Animal {
 void makeSound() {
 System.out.println("Dog barks");
 }
}

class Test {
 public static void main(String[] args) {
 Animal myDog = new Dog();
 myDog.makeSound();
 }
}
```



}

- a) "Animal makes a sound"
- b) "Dog barks"
- c) "Animal makes a sound" followed by "Dog barks"
- d) "Compilation Error"

15. What determines which method gets called in dynamic method dispatch? [      ]

- a) The reference type of the object.
- b) The actual object type (runtime type).
- c) The method signature in the reference class.
- d) The method name alone.

### Module-3: Interfaces

1. What is an interface in Java? [      ]

- a) A class that provides a default implementation of methods.
- b) A class that can extend multiple other classes.
- c) A reference type that can contain only abstract methods and static final variables.
- d) A type that can contain method implementations and instance variables.

2. Which keyword is used to define an interface in Java? [      ]

- a) interface
- b) implements
- c) abstract
- d) class

3. Can an interface in Java extend another interface? [      ]

- a) No, interfaces cannot extend other interfaces.
- b) Yes, but it can only extend one interface.
- c) Yes, an interface can extend multiple interfaces.
- d) Yes, but only if it is declared as abstract.

4. What happens if a class implements two interfaces that contain methods with the same signature? [      ]

- a) The class must provide its own implementation of the method.
- b) The class must choose one implementation provided by one of the interfaces.
- c) The class cannot compile due to ambiguity.
- d) The compiler automatically provides a default implementation.

5. Which of the following is true about default methods in interfaces ? [      ]
- a) Default methods can be overridden by classes that implement the interface.
  - b) Default methods cannot be overridden by classes that implement the interface.
  - c) Default methods must be static.
  - d) Default methods are abstract by default.
6. Can an interface have instance variables in Java? [      ]
- a) Yes, but only if they are declared as static.
  - b) No, interfaces cannot have instance variables.
  - c) Yes, but they must be declared as final.
  - d) Yes, but they must be private.
7. Which of the following methods are implicitly abstract in an interface? [      ]
- a) Static methods                      b) Default methods
  - c) Instance methods                  d) Private methods
8. If a class implements multiple interfaces with conflicting default methods, which method implementation will be used? [      ]
- a) The implementation from the first interface listed.
  - b) The implementation from the second interface listed.
  - c) The implementation must be explicitly overridden in the class.
  - d) The compiler will automatically resolve the conflict.
9. Which access modifier is allowed for methods in an interface? [      ]
- a) private                              b) protected
  - c) public                                d) default
10. Which of the following can be a valid method signature in an interface? [      ]
- a) void method1(int a, int b);      b) private void method2();
  - c) protected void method3();      d) static void method4();
11. What is an interface in Java? [      ]
- a) A class that provides a default implementation of methods.
  - b) A class that can extend multiple other classes.
  - c) A reference type that can contain only abstract methods and static final variables.
  - d) A type that can contain method implementations and instance variables.

12. Which keyword is used to define an interface in Java? [      ]
- a) interface                                      b) implements  
c) abstract                                        d) class
13. Can an interface in Java extend another interface? [      ]
- a) No, interfaces cannot extend other interfaces.  
b) Yes, but it can only extend one interface.  
c) Yes, an interface can extend multiple interfaces.  
d) Yes, but only if it is declared as abstract.
14. What happens if a class implements two interfaces that contain methods with the same signature? [      ]
- a) The class must provide its own implementation of the method.  
b) The class must choose one implementation provided by one of the interfaces.  
c) The class cannot compile due to ambiguity.  
d) The compiler automatically provides a default implementation.
15. Which of these can be used to fully abstract a class from its implementation? [      ]
- a) Objects                                      b) Packages  
c) Interfaces                                    d) None of the Mentioned
16. Which of the following is the correct way of implementing an interface salary by class manager? [      ]
- a) class manager extends salary {}  
b) class manager implements salary {}  
c) class manager imports salary {}  
d) none of the mentioned
17. What will be the output of the following Java program? [      ]
- ```
interface calculate
{
    void cal(int item);
}
class display implements calculate
{
    int x;
```

```
public void cal(int item)
{
    x = item * item;
}
}
class interfaces
{
    public static void main(String args[])
    {
        display arr = new display;
        arr.x = 0;
        arr.cal(2);
        System.out.print(arr.x);
    }
}
```

- a) 0 b) 2 c) 4 d) None of the mentioned

18. How do you access a static method of an interface?

[]

- a) Using the interface name
- b) Using the method name directly
- c) Through an object of the interface
- d) Through an implementation class

Part-B**Short Answer Questions (2M)****Module 1: Arrays**

1. How do you declare and initialize an array of integers in Java with the values 1, 2, 3, 4, and 5? [BL2]
2. What will be the output of the following code snippet?
 - i. `int[] arr = new int[3];`
 - ii. `System.out.println(arr[2]);` [BL3]
3. How can you find the length of an array in Java? [BL2]
4. What is the difference between `array.length` and `array.length()` in Java? [BL2]
5. How do you access the last element of an array `arr` in Java? [BL2]
6. Write a java program to add and remove elements in Array list. [BL2]

Module 2: Inheritance

1. What is inheritance in Java? [BL2]
2. How do you declare a subclass in Java? [BL2]
3. What is method overriding? [BL2]
4. Can a subclass call the constructor of its super class? If so, how? [BL2]
5. What is the purpose of the `super` keyword in Java? [BL2]
6. What is the purpose of the `Final` keyword in Java? [BL2]
7. What is Dynamic Method Dispatch? [BL2]

Module 3: Interfaces

1. What is the purpose of an interface in Java? [BL2]
2. Can an interface extend multiple other interfaces? If so, how? [BL3]
3. What is the difference between an abstract class and an interface in Java? [BL2]
4. How do you implement an interface in a class? [BL3]
5. Can an interface have instance variables in Java? [BL3]
6. Can we define a class within interface? Explain with suitable program. [BL3]
7. Write a Java program to implement nested Interface. [BL3]

Part-C**Descriptive Questions (5M)****Module 1: Arrays**

1. Write a Java program to find the largest number in an integer array. [BL3]
2. How do you create an array of 10 integers and initialize all elements to zero? Write a program. [BL3]
3. Write a Java program to find the sum of all elements in an integer array. [BL2]
4. What is a 2D array in Java? Give a simple example of how to declare and initialize a 2D array. [BL2]
5. Write a Java program to count the number of elements in an integer array. [BL3]
6. List different types of Inheritance. Explain with example programs. [BL2]

Module 2: Inheritance

1. Explain the concept of inheritance in Java and how it promotes code reusability. Provide an example to illustrate how a subclass can inherit fields and methods from a super class. [BL3]
2. Describe how constructor chaining works in inheritance. How does the constructor of a subclass invoke the constructor of its superclass? Include a code example to demonstrate this process. [BL3]
3. Discuss method overriding in Java. How does it differ from method overloading? Explain with an example how a subclass can provide a specific implementation of a method that is already defined in its super class. [BL3]
4. What is the super keyword used for in Java inheritance? Explain its role in accessing super class members and invoking super class methods, and provide examples of its usage in different scenarios. [BL3]
5. How can you prevent a class from being sub classed in Java? Discuss the use of the final keyword in this context and provide an example where a final class cannot be extended. [BL3]

Module 3: Interfaces

1. Explain what an interface is in Java and describe its primary purpose. How does an interface differ from an abstract class? Provide examples to illustrate the differences. [BL3]
2. Discuss how multiple inheritance is achieved in Java through interfaces. How can a class implement multiple interfaces, and what are the benefits of this approach? Provide a code example. [BL3]
3. Describe the role of default methods in interfaces. How do default methods differ from abstract methods, and when might you use them? Illustrate with a code example. [BL3]
4. Explain the concept of interface inheritance in Java. How can one interface extend another interface, and what are the implications for implementing classes? Provide an example to demonstrate this. [BL3]
5. How does Java handle the implementation of methods in an interface when a class implements multiple interfaces with conflicting method signatures? Explain with a code example how a class resolves such conflicts. [BL3]
6. An interface Number is defined in the following program. You have to declare a class A, which will implement the interface Number. Note that the method findSqr(n) will return the square of the number n. [BL3]

Testcase	Input	Output
1	-3	9
2	10	100

7. Create an interface “A” which contains methods add(int a, int b) and sub(int a, int b). [BL3]
Create an interface “B” which contains methods mul(int a, int b) and div(int a, int b).
Create a class called “Demo” which implements above two interfaces.

Part-D**Competitive Examination Questions**

1. Consider the following Java code:

```
public class ArrayTest {  
    public static void main(String[] args) {  
        int[] arr = new int[5];  
        for (int i = 0; i < arr.length; i++) {  
            arr[i] = i * 2;  
        }  
        System.out.println(arr[4]);  
    }  
}
```

What will be the output of the program?

2. Given the following Java classes:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```

What will be the output of the following code?

3. Given the following Java interface and class:

```
interface Vehicle {  
    void start();  
    default void stop() {  
        System.out.println("Vehicle stops");  
    }  
}  
  
class Car implements Vehicle {  
    public void start() {  
        System.out.println("Car starts");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Vehicle v = new Car();  
        v.start();  
        v.stop();  
    }  
}
```

What will be the output of the following code?

UNIT-4

Packages & Exception Handling

Modules

1. Packages and Java Library
2. Exception Handling

Objective:

- To learn how to use packages, exception handling in Java applications.

Syllabus:

Packages and Java Library: : Introduction, Defining Package, Importing Packages and Classes into Programs, Path and Class Path, Access Control.

Exception Handling: Introduction, Hierarchy of Standard Exception Classes, Keywords throws and throw, try, catch, and finally Blocks, Multiple Catch Clauses, Class Throwable, Unchecked Exceptions, Checked Exceptions.

Learning Outcomes:

Students will be able to

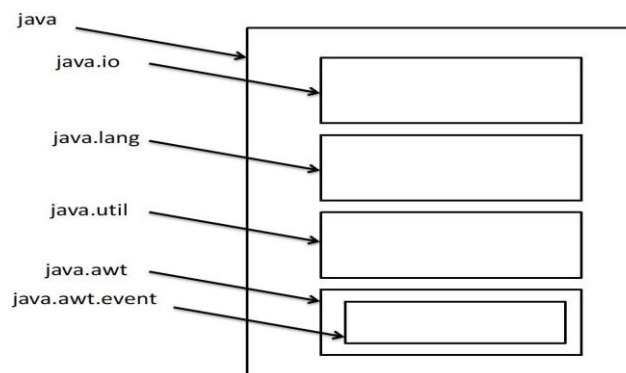
- make use of Packages and Java Libraries java application development.
- apply exception handling mechanisms in java application development.

Module 1: Packages and Java Library

1.1 Introduction

Packages:

A Package is a directory that contains classes and interfaces.



Advantages of Packages:

- Packages are useful to arrange group of classes and interfaces into a group. This puts together all the classes and interfaces performing the same task in the same package. For

example, `java.io` package contains all the classes and interfaces performing input and output operations.

- Packages hide the classes and interfaces into a sub directory. So, that accidental deletion of classes and interfaces will not take place.
- The classes and interfaces of a package are isolated from the classes and interfaces in another package. This means that you can use same name of a class of two different packages.
- A group of packages is called a library. The classes and interfaces of a package are like books in a library and can be reused several times.

Types of packages:

There are two types of packages.

1. Built-in packages
2. User-defined packages

1. Built-in packages

- ***java.lang***: `lang` stands for language. This package got primary classes and interfaces essential for developing a basic Java program.
- ***Java.util***: `util` stands for utility. This package contains useful classes and interfaces like `Stack`, `LinkedList`, `Hashtable`, `Vector`, `Arrays`, etc. These classes are called collections. There are also classes for handling date and time operations.
- ***java.io***: `io` stands for input and output. This package contains streams. A stream represents flow of data from one place to another place.
- ***java.awt***: `awt` stands for abstract window toolkit. This package helps to develop GUI (Graphics User Interface) where programs with colorful screens, paintings and images etc., can be developed.
- ***javax.swing***: This package helps to develop GUI like `java.awt`. The 'x' in `javax` represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, ***javax.swing*** is an extended package of ***java.awt***.
- ***java.net***: `net` stands for network. Client-Server programming can be done by using this package.
- ***java.applet***: Applets are programs which come from a server into a client and get executed on the client machine on a network. Applet class of this package is useful to create and use applets.
- ***java.text***: This package has two important Classes, ***DateFormat*** to format dates and times, and ***NumberFormat*** which is useful to format numeric values.
- ***java.sql***: `sql` stands for structured query language. This package helps to connect to databases like Oracle or Sybase, retrieve the data from them and use it in a Java program.

2. User-defined packages

Just like the Built-in packages shown earlier, the users of the Java language can also

create their own packages. They are called user-defined packages. User-defined packages can also be imported into other classes and used exactly in the same way as the Built-in packages.

1.2 Defining Package

To create a package the keyword “package” is used as:

```
package packname;  
package packname.subpackname;
```

- Write a program to create package with name pack and create class Addition.

```
package pack;  
public class Addition  
{  
    public void add(int a,int b)  
    {  
        int c=a+b;  
        System.out.println("The sum is "+c);  
    }  
}
```

- To compile the program by using following command,

```
javac -d . Addition.java
```

- The -d option tells the Java compiler to create a separate sub directory and place the class file there. The dot (.) after -d indicates that the package should be created in the current directory.
- From the above statement our package with “*Addition.class*” is ready. The next step is we have use the *add()* method in the program. For this purpose, we have to create another class and import that package.
- Write a java program to how to use Addition class of a package pack.

```
import pack.Addition;  
class DemoPack  
{  
    public static void main(String[] args)  
    {  
        Addition a1=new Addition();  
        a1.add(15,27);  
    }  
}
```

Output: The Sum is 42

Example: Write a program to create package with name pack and create class Calculator and use them another class.

```
package pack;  
public class Calculator  
{
```

```
public void add(int a,int b)
{
    int c=a+b;
    System.out.println("The Sum is "+c);
}
public void sub(int a,int b)
{
    int c=a-b;
    System.out.println("The Subtraction is "+c);
}
public void mul(int a,int b)
{
    int c=a*b;
    System.out.println("The Multiplication is "+c);
}
public void div(int a,int b)
{
    int c=a/b;
    System.out.println("The Division is "+c);
}
}
```

Output: *javac -d . Calculator.java*

1.3 Importing Packages and Classes into Programs

All the java programs need to refer one or more packages and classes in the source code. If a program needs several classes of a package, it is better to import the whole package.

```
import pack.Calculator;
class DemoPack2
{
    public static void main(String[] args)
    {
        Calculator c=new Calculator();
        c.add(15,27);
        c.sub(35,5);
        c.mul(15,22);
        c.div(45,5);
    }
}
```

Output: *javac DemoPack2.java*

```
java DemoPack2
The Sum is 42
The Subtraction is 30
```

The Multiplication is 330

The Division is 9

If the package pack is not available in the current directory, then what happens? Suppose our program is running in *D:* and the package available in *C:\sub*. In this case, the compiler should be given information regarding the package location by mentioning the directory of the package in class path.

1.4 Path and Class Path

When the java programs to be run on command prompt, the programmer must know how to set PATH and CLASSPATH.

PATH: The PATH is an environment variable used by the operating system to find the executive binary files like javac, which compiles the source code into byte code and java, which interprets the byte code through JVM for execution of the program and other such commands. PATH tells the system where to locate the JDK files. PATH may be set in two ways

1. Set path on command prompt for individual program

C:\Program Files\Java\jdk1.8.0\bin

Set path=%path%; C:\Program Files\Java\jdk1.8.0\bin

2. Set the PATH as environmental variables to execute any number of programs.

CLASSPATH:

The CLASSPATH is an environment variable that tells the Java compiler where to look for class files to import. CLASSPATH is generally set to a directory or a JAR (Java Archive) file.

- To see what is there in currently in CLASSPATH variable in your system. You can type the command in windows.

echo %CLASSPATH%

- Suppose, preceding command has displayed class path as:

C:\rnr;.

- This means the current class path is set to rnr directory in C:\ and also to the current directory represented by dot (.). Our package pack does not exist in either rnr or current directory. Our package exists in D:\sub, as:

set CLASSPATH=D:\sub;.;%CLASSPATH%

1.5 Access Control

Access Modifier	within class	within package	outside package by subclass only	outside package
private	YES	NO	NO	NO

<i>default</i>	YES	YES	NO	NO
<i>protected</i>	YES	YES	YES	NO
<i>public</i>	YES	YES	YES	YES

Module 2: Exception Handling

2.1 Introduction

A software engineer may also commit several errors while designing the project or developing the code. These errors are also called ‘bugs’ and the removing them are called ‘debugging’.

Errors:

There are basically three types of errors in java.

- a) Compile-time errors
- b) Logical errors
- c) Runtime errors

a) Compile-time errors

These are syntactical errors found in the code, due to which a program fails to compile. For example, forgetting a colon in the statements like if, while, for, def, etc. will result in compile-time error. Such errors are detected by java compiler and the line number along with error description is displayed by the java compiler.

b) Logical errors

These errors depict flaws in the logic of the program. The programmer might be using a wrong formula or the design of the program itself is wrong. Logical errors are not detected either by java compiler or JVM. The programmer is solely responsible for them.

c) Runtime errors

These errors represent inefficiency of the computer system to execute a particular statement. For example, insufficient memory to store something or inability of the microprocessor to execute some statement come under run-time errors.

Example:

```
class Division
{
    public static void main(String[] args)
    {
        System.out.println("WELCOME");
    }
}
```

Output:

Exception in thread "main" java.lang.NoSuchMethodError: main

Note: Runtime errors are not detected by the java compiler. They are detected by the JVM, only at runtime.

Exceptions:

- An exception is a runtime error. All exceptions that occur only by runtime but some exceptions are detected at compile time and some others at runtime.
- The exceptions that are checked at compilation time by the java compiler are called 'checked exceptions'.
- The exceptions that are checked by the JVM are called 'unchecked exceptions'.
- Unchecked exceptions and errors are considered as unrecoverable and the programmer cannot do anything when they occur. The programmer can write a java program with unchecked exceptions and errors and can compile the program. He can see their effect only when he runs the program.
- So, java compiler allows him to write a java program without handling the unchecked exceptions and errors.
- Checked exceptions should either handle them or throw them without handling them. He cannot simply ignore them, as java compiler will remind him of them.

Example:

```
import java.io.*;
import java.lang.*;
class Test {
    public static void main(String[] args)
    {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);
        System.out.print("Enter a single Character: ");
        char ch = (char)br.read();
        System.out.print("\n The Character is "+ch);
    }
}
```

Output:

javac Test.java

```
Test.java:9: unreported exception java.io.IOException; must be caught or
declared to be thrown
```

```
        char ch = (char)br.read();
                        ^
```

1 error

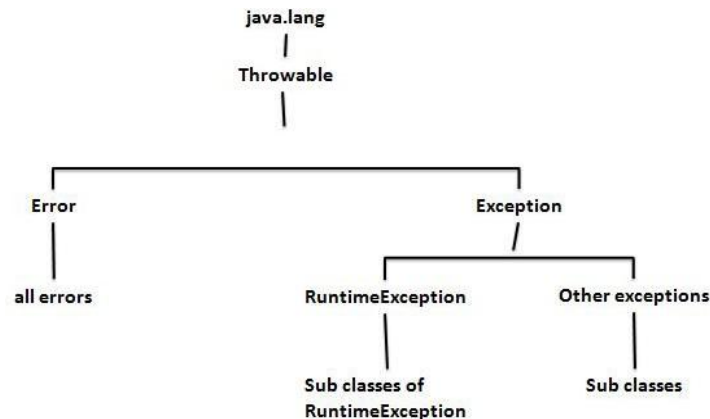
- Here, at compilation-time Java Compiler shows an IOException i.e., Checked Exception. It must be caught or declared to be thrown. So, we throw it out of **main()** method without handling it. This is done by throws clause written after **main()** method in the above program, as:


```
public static void main(String[] args) throws IOException
```

- All Exceptions are declared as classes in java. Even the errors are also represented by classes. All these classes are descended from a super class called *Throwable*. And this class belongs to *java.Lang* package.

2.2 Hierarchy of Standard Exception Classes

An exception is an error which can be handled. It means when an exception happens, the programmer can do something to avoid any harm. But an error is an error which cannot be handled, it happens and the programmer cannot do anything.



2.3 Keywords throws and throw

throws clause:

Even if the programme is not handling runtime exceptions, the Java compiler will not give any error related to runtime exceptions. But the rule is that the programmer should handle checked exceptions. In case the programmer does not want to handle the checked exceptions, he should throw them out using throws clause. Otherwise, there will be an error flagged by Java compiler.

Example: Write a program that shows the use of throws clause.

```
import java.io.*;
import java.lang.*;
class Test {
    public static void main(String[] args) throws IOException
    {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);
        System.out.print("Enter the String: ");
        String str = br.readLine();
        System.out.print("\n The String is "+str);
    }
}
```

throw clause:

There is also a throw statement available in Java to throw an exception explicitly and

catch it. Let us see how it can be used. In the following program, we are creating an object of `NullPointerException` class and throwing it out of try block, as shown here:

Example: Write a program that shows the use of `throw` clause.

```
import java.util.*;
class ThrowException
{
    public static void main(String[] args)
    {
        try{
            System.out.println("WELCOME");
            throw new NullPointerException("Exception Data");
        }
        catch(NullPointerException ne)
        {
            System.out.println(ne);
        }
    }
}
```

Output:

```
WELCOME
java.lang.NullPointerException: Exception Data
```

2.4 try, catch and finally blocks

When there is an exception, the user data may be corrupted. This should be tackled by the programmer by carefully designing the program. For this, he should perform the following 3 steps:

Step 1: The programmer should observe the statements in his program where there may be a possibility of exceptions. Such statements should be written inside a **try** block. A **try** block looks like as follows:

```
try{
    statements;
}
```

The greatness of try block is that even if some exception arises inside it, the program will not be terminated. When JVM understands that there is an exception, it stores the exception details in an exception stack and then jumps into a catch block.

Step 2: The programmer should write the Catch block where he should display the exception details to the user. This helps the user to understand that there is some error in the program. The programmer should also display a message regarding what can be done to avoid this error. **catch** block looks like as follows:

```
catch(ExceptionClass ref)
{
    Statements;
}
```

Step 3: Lastly, the programmer should perform clean up operations like closing the files a terminating the threads. The programmer should write this code in the **finally** block. **final**

block looks like as follows:

```
finally{  
    Statements;  
}
```

Example: Write a program by using try, catch and finally block.

```
class Division  
{  
    public static void main(String[] args)  
    {  
        try{  
            System.out.println("WELCOME");  
            int a=5;  
            int b=0;  
            int c=a/b;  
            System.out.println("The Division is "+c);  
        }  
        catch(ArithmeticException ae)  
        {  
            System.out.println("Division with zero is not possible");  
        }  
        finally{  
            System.out.println("LOGOUT");  
        }  
    }  
}
```

Output:

```
WELCOME  
Division with zero is not possible  
LOGOUT
```

2.5 Multiple Catch Clauses

Most of the times there is possibility of more than one exception present in the program. In this case, the programmer should write multiple catch blocks to handle each one of them.

Example: write a java program to handle multiple exceptions.

```
import java.util.*;
class Division
{
    public static void main(String[] args)
    {
        try{

            System.out.println("WELCOME"); Scanner
            sc=new Scanner(System.in);
            System.out.print("Enter a value: "); int
            a=sc.nextInt();
            System.out.print("Enter b value: "); int
            b=sc.nextInt();
            int c=a/b;
            System.out.println("The Division is "+c);
        }

        catch(InputMismatchException ae)
        {
            System.out.println("Wrong Input");
        }
        catch(ArithmeticException ae)
        {
            System.out.println("Division with zero is not possible");
        }
        finally{
            System.out.println("LOGOUT");
        }
    }
}
```

Output-1:

```
WELCOME
Enter a value: 5
Enter b value: k
Wrong Input
LOGOUT
```

Output-2:

```
WELCOME
Enter a value: 5
Enter b value: k
Division with zero is not possible
LOGOUT
```

2.6 Class Throwable

The *Throwable* class is the superclass of every error and exception in the Java language. Only objects that are one of the subclasses this class are thrown by any “Java Virtual Machine” or may be thrown by the Java *throw* statement.

Class Declaration:

```
public class Throwable
    extends Object
```

implements Serializable

Example:

```
import java.io.*;
class MyClass{
    public static void main(String[] args)throws Exception
    {
        try {

            testException();
        }
        catch (Throwable e) {
            System.out.println("Exception: "
                               + e.toString());
        }
    }
    public static void testException()throws Exception
    {
        throw new Exception("New Exception Thrown");
    }
}
```

Output:

Exception:

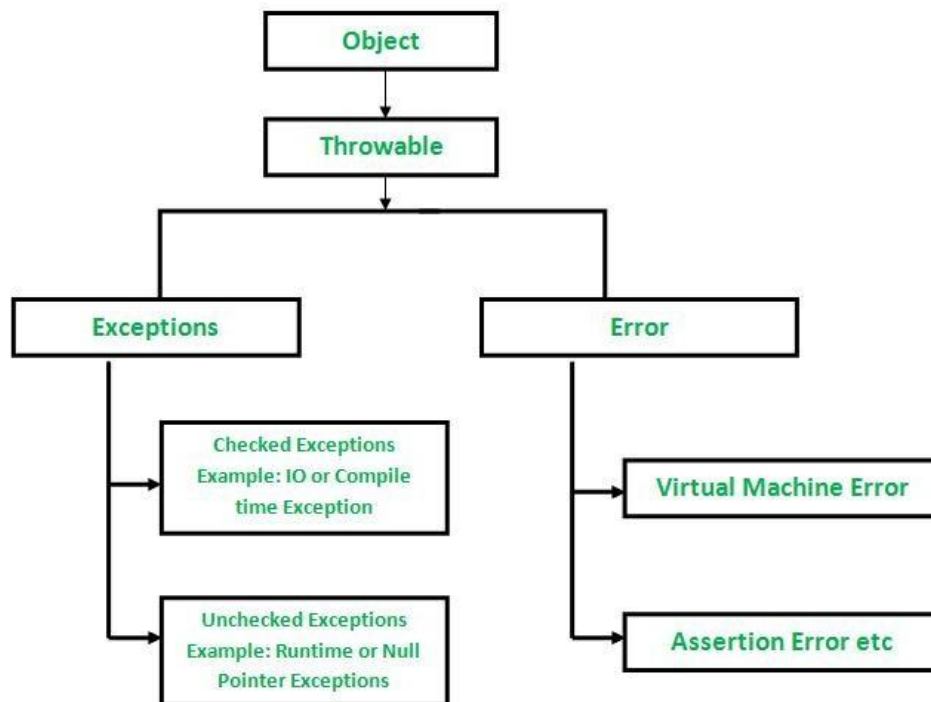
java.Lang.Exception:
New Exception Thrown

2.7 Checked and Unchecked Exceptions

In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions.

In Java, there are two types of exceptions:

- Checked exceptions
- Unchecked exceptions



2.7.1 Unchecked Exceptions

These are the exceptions that are not checked at compile time. In C++, all exceptions are unchecked, so it is not forced by the compiler's to either handle or specify the exception. It is up to the programmers to be civilized and specify or catch the exceptions. In Java, exceptions under Error and RuntimeException classes are unchecked exceptions, everything else under throwable is checked.

Consider the following Java program. It compiles fine, but it throws ArithmeticException when run. The compiler allows it to compile because ArithmeticException is an unchecked exception.

Example:

```
class MyClass{
    public static void main(String args[])
    {
        int x = 0;
```

```
        int y = 10;  
        int z = y / x;  
    }  
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Main.main(Main.java:5)  
Java Result: 1
```

2.7.2 Checked Exceptions

These are the exceptions that are checked at compile time. If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using the throws keyword. Checked exceptions can be fully checked or partially checked.

- **Fully Checked Exception:** A checked exception where all its child classes are also checked (e.g., *IOException*, *InterruptedException*).
- **Partially Checked Exception:** A checked exception where some of its child classes are unchecked (e.g., *Exception*).

Example:

```
class MyClass{  
    public static void main(String[] args) throws IOException  
    {  
        FileReader file = new FileReader("C:\\\\test\\\\a.txt");  
        BufferedReader fileInput = new BufferedReader(file);  
        for (int counter = 0; counter < 3; counter++)  
            System.out.println(fileInput.readLine());  
        fileInput.close();  
    }  
}
```

Types of Exceptions:

There are two types of Exceptions.

- a) Built-in Exceptions
- b) User-defined Exceptions

a) Built-in Exceptions

Exception	Description
<i>ArithmeticException</i>	Arithmetic error, such as divide-by-zero.
<i>ArrayIndexOutOfBoundsException</i>	Array index is out-of-bounds.
<i>IndexOutOfBoundsException</i>	Some type of index is out-of-bounds.
<i>NullPointerException</i>	Invalid use of a null reference.
<i>NumberFormatException</i>	Invalid conversion of a string to a numeric format.

<i>ClassNotFoundException</i>	Class not found.
<i>InterruptedException</i>	One thread has been interrupted by another thread.
<i>NoSuchMethodException</i>	A requested method does not exist.

Assignment cum Tutorial questions

Part-A

Objective Questions (1M)

Module 1: Packages and Java Library

- Which of these keywords is used to define packages in Java? []
a) pkg b) Pkg
c) package d) Package
- Which of this access specifier can be used for a class so that its members can be accessed by a different class in the same package? []
a) Public b) Protected
c) No Modifier d) All of the mentioned
- Which of these access specifiers can be used for a class so that its members can be accessed by a different class in the different package? []
a) Public b) Protected
c) Private d) No Modifier
- Which of the following is the correct way of importing an entire package 'pkg'? []
a) import pkg. b) Import pkg.
c) import pkg.* d) Import pkg.*
- Which of the following package stores all the standard java classes? []
a) lang b) java
c) util d) java.packages
- Which of the following package stores all the simple data types in java? []
a) lang b) java
c) util d) java.packages
- Which of these is a wrapper for data type int? []
a) Integer b) Long
c) Byte d) Double
- Which of these is a super class of wrappers Long, Character & Integer? []
a) Long b) Digits
c) Float d) Number
- Which of these class is a superclass of all other classes? []
a) Math b) Process
c) System d) Object
- Which of these method of Object class can generate duplicate copy of the object on which it is called? []

- a) clone() b) copy()
c) duplicate() d) dito()
11. Which of these standard collection classes implements a dynamic array? []
a) AbstractList b) LinkedList
c) ArrayList d) AbstractSet
12. Which of these standard collection classes implements a linked list data structure? []
a) AbstractList b) LinkedList
c) HashSet d) AbstractSet
13. Which of these method is used to add an element to the start of a LinkedList object? []
a) add() b) first()
c) AddFirst() d) addFirst()
14. Which of these methods can be used to delete the last element in a LinkedList object? []
a) remove() b) delete()
c) removeLast() d) deleteLast()
15. What will be the output of the following Java code? []

```
import java.util.*;  
class date  
{  
    public static void main(String args[])  
    {  
        Date obj = new Date();  
        System.out.print(obj);  
    }  
}
```


a) Prints Present Date b) Runtime Error
c) Any Garbage Value d) Prints Present Time & Date

Module 2: Exception Handling

1. Which of these keywords is not a part of exception handling? []
a) try b) finally
c) thrown d) catch
2. Which of these keywords must be used to monitor for exceptions? []
a) try b) finally
c) throw d) catch
3. Which of the following classes can catch all exceptions which cannot be caught? []
a) RuntimeException b) Error
c) Exception d) ParentException
4. Which of the following is a super class of all exception type classes? []
a) Catchable b) RuntimeExceptions
c) String d) Throwable
5. Which of the following keyword is used by a calling function to handle exception thrown by a called function? []
a) throws b) throw
c) try d) catch

-
6. Which of the following should be true of the object thrown by a thrown statement? []
- a) Should be assignable to String type
 - b) Should be assignable to Exception type
 - c) Should be assignable to Throwable type
 - d) Should be assignable to Error type
7. Which of these is a super class of all exceptional type classes? []
- a) String b) RuntimeExceptions
 - c) Throwable d) Cacheable
8. What exception thrown by `parseInt()` method? []
- a) `ArithmeticException` b) `ClassNotFoundException`
 - c) `NullPointerException` d) `NumberFormatException`
9. Which of these keywords is used to by the calling function to guard against the exception that is thrown by called function? []
- a) `try` b) `throw`
 - c) `throws` d) `catch`
10. A single try block must be followed by which of these? []
- a) `finally` b) `catch`
 - c) `finally & catch` d) none of the mentioned
11. Which of these exceptions handles the divide by zero error? []
- a) `ArithmeticException` b) `MathException`
 - c) `IllegalAccessExcepion` d) `IllegarException`
12. Which of these exceptions will occur if we try to access the index of an array beyond its length? []
- a) `ArithmeticException` b) `ArrayException`
 - c) `ArrayIndexException` d) `ArrayIndexOutOfBoundsException`
13. Which of these statements is incorrect? []
- a) try block need not to be followed by catch block
 - b) try block can be followed by finally block instead of catch block
 - c) try can be followed by both catch and finally block
 - d) try need not to be followed by anything
14. Which of these classes is used to define exceptions? []
- a) `Exception` b) `Throwable`
 - c) `Abstract` d) `System`
15. Which of these methods is used to print stack trace? []
- a) `obtainStackTrace()` b) `printStackTrace()`
 - c) `getStackTrace()` d) `displayStackTrace()`
-

PART B
Short Answer Questions (2 Marks)

Module 1: Packages

1. Define package in java. [BL2]
2. What is the importance of CLASSPATH [BL2]
3. Can I import same package/class twice? Will the JVM load the package twice at runtime? [BL3]

Module 2: Exception Handling

1. What is Exception. [BL2]
2. What is finally keyword in java. [BL2]
3. Differentiate compile time errors and runtime errors in java. [BL3]
4. Is it mandatory for a catch block to be followed after a try block? Explain. [BL3]
5. Will the final block be executed if the code System.exit(0) is written at the end of the try block? [BL4]

Part-C

Descriptive Questions (5M)

Module 1: Packages and java library

1. With sample program explain the creation of packages. Accessing a package and hiding classes with packages. [BL3]
2. What is the significance of the CLASSPATH environment variable in creating or using a package? [BL2]
3. How can we add a class to a package? Write about relative and absolute paths. [BL3]
4. Write the procedure to create package with multiple public classes. [BL3]
5. List and explain any four packages in Java language. [BL2]

Module 2: Exception Handling

1. What are the various types of exceptions available in Java? Also discuss on how they are handled? [BL2]
2. What is the role of 'finally' keyword in exception handling? Explain with an example. [BL2]
3. What is the importance of Exception Handling in Java? Define and distinguish between checked and unchecked exceptions [BL3]
4. Demonstrate nested try statements and finally statements. [BL3]
5. Give the syntax of exception handling and also handle exception occurred during the execution of divide by zero. [BL3]
6. Write a java program that illustrates the application of multiple catch statements. [BL3]
7. Is it possible to Rethrow exceptions? Justify your answer [BL3]

Part-D**Competitive Examination Questions**

1. What will be the output of the following Java program?

```
package pkg;
class display
{
    int x;
    void show()
    {
        if (x > 1)
            System.out.print(x + " ");
    }
}
class packages
{
    public static void main(String args[])
    {
        display[] arr=new display[3];
        for(int i=0;i<3;i++)
            arr[i]=new display();
        arr[0].x = 0;
        arr[1].x = 1;
        arr[2].x = 2;
        for (int i = 0; i < 3; ++i)
            arr[i].show();
    }
}
```

Note : packages.class file is in directory pkg;

- a) 0
- b) 1
- c) 2
- d) 0 1 2

2. What will be the output of the following Java code?

```
class exception_handling
{
    public static void main(String args[])
    {
        try
        {
            int a = 1;
            int b = 10 / a;
            try
            {
```

```
        if (a == 1)
            a = a / a - a;
        if (a == 2)
        {
            int c[] = {1};
            c[8] = 9;
        }
    }
    finally
    {
        System.out.print("A");
    }
}
catch (Exception e)
{
    System.out.println("B");
}
}
```

- a) A
- b) B
- c) AB
- d) BA

UNIT-5

Multithreading and Event Handling

Modules

1. Multithreading.
2. Event Handling.

Objective:

- To learn how to implement multithreading and
- developing GUI based applications through Event handling.

Syllabus:**Multithreading and Event Handling**

Multithreaded Programming: Introduction, Need for Multiple Threads, Multithreaded Programming for Multi-core Processor, Main Thread-Creation of New Threads, Thread Priority. Synchronization, Inter-thread Communication - Suspending, Resuming, and Stopping of Threads.

Event Handling- Events, event sources, event classes, event listeners, delegation event model, handling mouse and keyboard events.

Learning Outcomes:

Students will be able to

- make use of multithreading in java application development.
- apply event handling mechanisms in java application development.

Module 1: Multithreaded Programming

1.1 Introduction

A thread represents a separate path of execution of group of statements. In a java program, if we write a group of statements, then these statements are executed by JVM on by one. This execution is called a thread, because JVM uses a thread to execute these statements. This means that in every java program, there is always a thread running internally. This thread used by JVM to execute the program statements.

Program: write a java program to find the thread used by JVM to execute the statements.

```
class Demo
{
    public static void main(String[] args)
    {
```

```
        System.out.println("Welcome");  
        Thread t=Thread.currentThread();  
        System.out.println("Current Thread= "+t);  
        System.out.println("Thread Name= "+t.getName());  
    }  
}
```

Output: Welcome

Current Thread= Thread[main, 5, main]

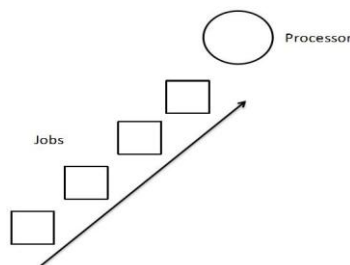
Thread Name= main

In the above program, *currentThread()* is a static method in Thread class. So we called it as *Thread.currentThread()*. It displayed as Thread[main, 5, main]. Here, the first main indicated the name of the thread running the current code. We get 5 which is a number representing the priority numbers will range from 1 to 10. 1 is minimum priority, and 10 is the maximum priority of a thread.

1.2 Need for Multiple Thread

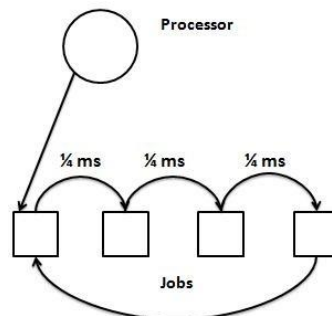
- The present speed of about 15 GHz is about the upper possible limit because beyond this, the cooling problems are tremendous.
- Further, increase in throughput of computer is possible only by dividing the program into segments that are data dependent and can be processed simultaneously by more than one processor;

- thus, it decreases the total time of computation.
 - This is the basis on which supercomputers are built.
 - In a supercomputer, thousands of processors are employed to concurrently process the data.
 - Hardware developers have gone a step further by placing more than one core processor in the same CPU chip.
 - Thus, now, *we have multi-core CPUs.*
- A thread is group of statements, and these statements are executed in two ways.
 - Single tasking
 - Multi tasking
 - **Single tasking:**
 - In single tasking only one task is given to the processor at a time. This means we are waiting lot of time and micro processor has to sit idle with doing any job for long time.



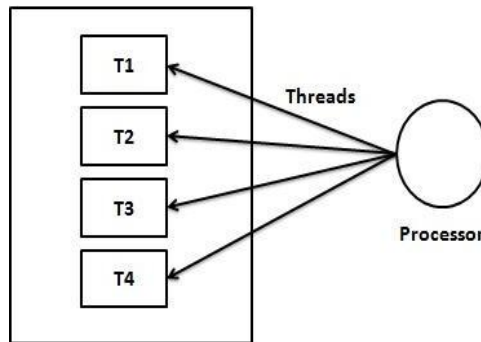
• **Fig: Single Tasking**

- **Multi Tasking:**
 - In multi tasking, several jobs are executed at a time. These jobs are executed by the processor simultaneously. One processor has executed the several tasks by using round robin method.



• **Fig: Process based Multi tasking**

- In multi tasking there are two types:
 - Process based Multi tasking
 - Thread based Multi tasking
- In process based multi tasking several programs are executed at a time by the microprocessor.
- In Thread Based Multi tasking several parts of programs are executed by the processor.

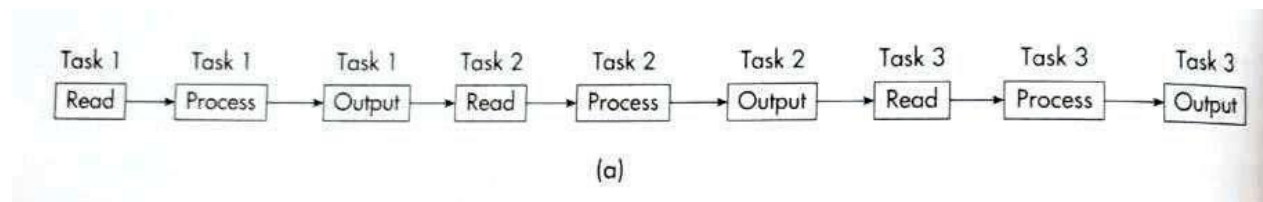


- **Note:** Threads are light weight process because they utilize minimum resources of the system. This means they take less memory and less processor time.

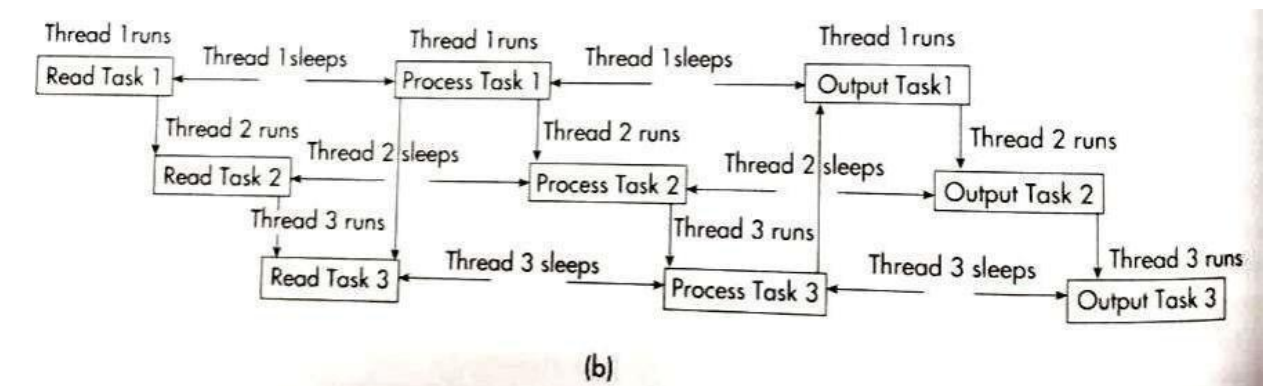
1.3 Multithreaded Programming for Multi-core Processor

- A CPU may have two cores - dual core or four cores - quad, six cores, or more.
- CPUs having as many as 50 cores have also been developed.
- Moreover, computers with multi-core CPU are affordable and have become part of common man's desktop computer.
- Advancements in hardware are forcing the development of suitable software for optimal utilization of the processor capacity. *Multithread processing is the solution.*
- Multithread programming is inbuilt in Java and CPU capacity utilization may be improved by having multiple threads that concurrently execute different parts of a program.
- How Single Processor – tasks carried out in Single Sequence is

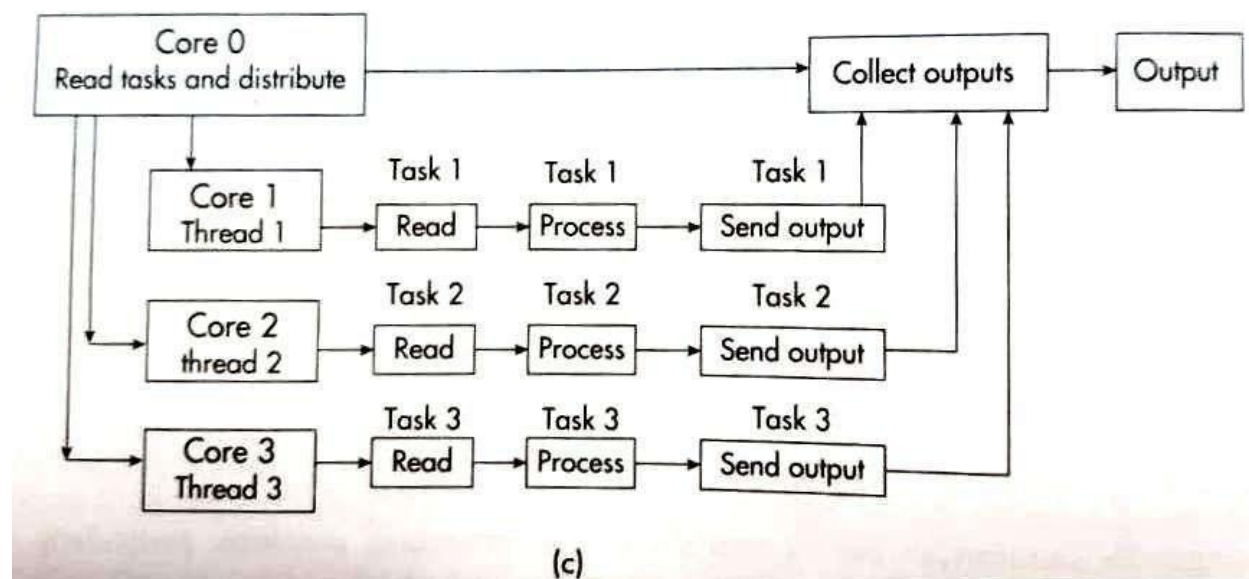
illustrated in the following diagram.



A. The following diagram shows the Single processor – threads share the time of processor



B. The following diagram shows the Multi-core processor – threads execute concurrently



Thread Class

In Java, threads are based on the class Thread belonging to java.lang package, that is,

java.lang.Thread.

A thread is an object of class Thread and can invoke the instance methods defined in the class.

Even if a thread is not explicitly defined, one thread is automatically created by the system for the execution of the main method. **It is generally called main Thread.**

A thread does not start working on birth. It has to invoke the start() method that gives it the life, otherwise it is a lifeless thread object.

After getting life, a thread executes a code of instructions (target) as specified by the user in the overloaded method run().

The Thread class has defined several constructors.

i. **Thread():** It allocates a new thread object as thread(null, null, generatedname).

Every thread must have a name.

ii. **Thread(String threadname):** It allocates a thread with name specified by the user. It is of the form thread(nall null, name). A thread may be created as

Thread t2 new Thread("MyThread");

iii. **Thread(Runnable object) :** The constructor allocates a thread with a specified target.

The name by the compiler as Thread-0, Thread-1, and so on.

iv. **Thread (Runnable object, String threadName):** Thread is allocated with a specified target and user specified name threadnume.

v. **Thread (ThreadGroupgroup, Runnable object):** It allocates thread with specified group and target.

vi. **Thread (ThreadGroupgroup, Runnable object, String Threadname):** The constructor allocates thread with specified thread group, target, and thread name.

Example: ThreadX.java

```
public class ThreadX extends Thread
{
    public void run()
    {
        System.out.println("It is Threadx class");
    }
    public static void main(String args[])
    {
        Thread a= new Thread (new ThreadX(), "FirstThread");
        Thread b= new Thread (new ThreadX());
        System.out.println("Name of a = " + a.getName());
        System.out.println("Name of b = "+ b.getName());
        th.start();
        t1.start();
    }
}
```

Output:

```
C:\ >javac ThreadX.java
```

```
C:\ >java ThreadX
```

```
Name of a = FirstThread
```

```
Name of b = Thread-2
```

```
It is Threadx class
```

```
It is Threadx class
```

1.4 Main Thread

When we run a program in Java, one thread is automatically created and it executes the main method. The thread is called main thread. This is the thread from which other threads are created.

- 1.4.1 The threads created from the main thread are called child threads.
- 1.4.2 A program keeps running as long as the user threads are running.
- 1.4.3 The main thread also has the status of a user thread.
- 1.4.4 The child threads spawned from the main thread also have the status of user thread.
- 1.4.5 Therefore, main thread should be the last thread to finish so that the program finishes when the main method finishes.
- 1.4.6 A thread is controlled through its reference like any other object in Java.
- 1.4.7 The main thread can also be controlled by methods of Thread class through its reference that can be obtained by using the static method `currentThread()`. Its signature is
`Thread thread = Thread.currentThread();`

`sleep()` method suspend the execution for the given specified time interval.

Ex: `thread.sleep(500);`

the above statement will suspend the execution of `main()` method for 500 ms, which is the argument of the method.

`setName()` method add the new name to the existing thread

Ex: `thread.setName("My Thread")`

Illustration of main thread and methods `setName()` and `getName()`

Example: MyThread.java

```
class MyThread
{
```

```
public static void main (String args[])
{
    Thread thread = Thread.currentThread(); //Thread reference
    System.out.println("CurrentThread :" + thread);
    System.out.println("Before modification ,Thread Name =" +
        thread.getName());
    System.out.println("Change the name to MyThread.");
    thread.setName("MyThread"); //new name for main thread
    System.out.println("After modification ,Thread Name =" +
        thread.getName());
    //try block contains code to be executed by main thread
    try
    {
        for (int i=0; i<4; i++)
        {
            boolean B = thread.isAlive();
            System.out.println("Is the thread alive? " + B);
            System.out.println(Thread.currentThread()+ " i = "+ i);
        }
    }
}
```

```
        Thread.sleep(1000);  
    }  
}catch(InterruptedException e)  
{  
    System.out.println("Main thread interrupted");  
}  
}  
}
```

Output:

```
C:\>javac MyThread.java
```

```
C:\>java MyThread
```

```
CurrentThread :Thread[main,5,main]
```

```
Before modification ,Thread Name =main
```

```
Change the name to MyThread.
```

```
After modification ,Thread Name =MyThread
```

```
Is the thread alive? true
```

```
Thread[MyThread,5,main] i = 0
```

```
Is the thread alive? true
```

```
Thread[MyThread,5,main] i = 1
```

```
Is the thread alive? true
```

```
Thread[MyThread,5,main] i = 2
```

```
Is the thread alive? true
```

```
Thread[MyThread,5,main] i = 3
```

1.5 Creation of New Threads

The new thread is created by creating a new instance of Thread class. The enhancements in Java 8 enables us to create and execute new threads in the following ways

- i. By extending Thread class
- ii. By implementing Runnable interface:

i. Creation of New Thread by Extending Thread Class

A thread is an object of class Thread that contains methods to control the behaviour of threads. **A class** that **extends** the class **Thread** inherits all methods and constructors

The procedure followed in this case is given as follows:

- 1) A class is declared that extends Thread class. Since this is a subclass, it inherits the methods of Thread class.
- 2) This class calls a constructor of Thread class in its constructor.
- 3) In this class, the method run() is defined to override the run() method of Thread class.

The method run() contains the code that the thread is expected to execute.

- 4) The object of class the method start() inherited from Thread class for the execution of run().

Example: MyClass.java

```
public class MyClass extends Thread
{
    MyClass()
    {
        super("MyThread"); // constructor of Myclass
        start();
    }

    //The run() method is defined below
    public void run()
```



```
{  
System.out.println("It is MyThread.");  
}  
  
public static void main(String args[])  
{  
new MyClass(); //creates a new instance of MyClass  
}  
}
```

Output:

C:\>javac MyClass.java

C:\>java MyClass

It is MyThread.

ii. Creation of New Threads by Implementing Runnable interface

The runnable is an interface that has only one method in it, that is

```
public interface Runnable( public void run());
```

The full definition of method run() is included in the class. The thread begins with execution of run() and ends with end of run() method. The step wise procedure is given here.

1. Declare a class that implements Runnable interface.
2. Define the run() method in the class. This is the code that the thread will execute.
3. Declare an object of Thread class in main() method.
4. Thread class constructor defines the thread declared with operator new and the Runnable object is passed on to the new thread constructor.
5. Method start() is invoked by the thread object created.

The following Program illustrates a simple example of implementing Runnable.

Example: MyThreadClass.java

```
public class MyThreadClass implements Runnable
```

```
{  
public void run()  
{  
System.out.println("This is Runnable implementation.");  
}  
public static void main(String args[])  
{  
Thread Th = new Thread(new MyThreadClass());  
  
Th.start();  
  
}  
}
```

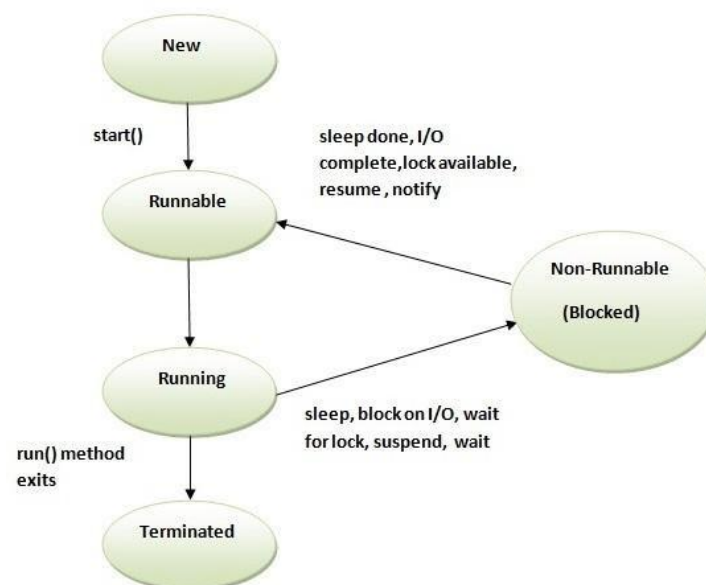
Output

This Is Runnable implementation.

Thread States

Thread Life Cycle:

1. **New:** The thread is in new state if you create an instance of Thread class but before the invocation of start() method.
2. **Runnable:** The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.
3. **Running:** The thread is in running state if the thread scheduler has selected it.
4. **Non-Runnable (Blocked):** This is the state when the thread is still alive, but is currently not eligible to run.
5. **Terminated:** A thread is in terminated or dead state when its run() method exits.



1.6 Thread Priorities:

Each thread has a priority. Priorities are represented by a number between 1 and 10. In most cases, thread scheduler schedules the threads according to their priority (known as pre-emptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

3 constants defined in Thread class:

1. public static int MIN_PRIORITY
2. public static int NORM_PRIORITY
3. public static int MAX_PRIORITY

Program:

```
class TestMultiPriority extends Thread
{
    public void run()
    {
        Thread t=Thread.currentThread();
        System.out.println("running thread name is:"+t.getName());
        System.out.println("running thread priority is:"+t.getPriority());
    }
    public static void main(String args[])
    {
        TestMultiPriority m1=new
        TestMultiPriority();      TestMultiPriority
        m2=new TestMultiPriority(); Thread t1=new
        Thread(m1);
        Thread t2=new Thread(m2);
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
    }
}
```

1.7 Thread Synchronization:

Synchronization in java is the capability *to control the access of multiple threads to any shared resource*. Java Synchronization is better option where we want to allow only one thread to access the shared resource.

In several multithreaded programs, the different threads are required to access the same resource, for example, a memory object during the execution. One thread may attempt to update it, while the other wants to read it and a third may try to alter the content of the memory.

Such a situation may give rise to *race condition* and the result may be quite different if the sequence of operations actually followed is different from the desired one.

Proper execution requires that at any point of time, only one thread be allowed to use the critical resource till it finishes its task.

We are all aware of the computerized banking system. In one branch, money may be deposited in your account, while you are withdrawing money at another branch. There can be a problem if one thread has partly finished its task, while the other gets in. It will corrupt the data.

Synchronization solves this problem by allowing only one thread can access the resource.

The second thread should be allowed only when the first has finished its task.

Example: SyncDemo.java

```
class Counter
{
    int count=0;
    public void increment()
    {
        count++;
    }
}
public class SyncDemo
{
    public static void main(String[] args) throws Exception
    {
        Counter c = new Counter();
        Thread t1 = new Thread(new Runnable()
        {
            public void run()
```

```
{  
for(int i=1;i<=5000; i++)  
{  
c.increment();  
}  
}  
});  
Thread t2 = new Thread(new Runnable()  
{  
public void run()  
{  
for(int i=1;i<=5000; i++)  
{  
c.increment();  
}  
}  
});  
t1.start();  
t2.start();  
t1.join();  
t2.join();  
System.out.println("Count = "+ c.count);  
}
```

Output:

C:\>javac SyncDemo.java

C:\>java SyncDemo

```
Count = 8343
```

```
C:\>java SyncDemo
```

```
Count = 9998
```

```
C:\>java SyncDemo
```

```
Count = 9865
```

```
C:\>java SyncDemo
```

```
Count = 9989
```

```
C:\>java SyncDemo
```

```
Count = 9790
```

```
C:\>java SyncDemo
```

```
Count = 9954
```

```
C:\>java SyncDemo
```

```
Count = 9799
```

Here the count should be 10000, but it is printing less than 10000. To solve this problem, add synchronized keyword to the increment() method as shown in the following code.

1.8 Inter-thread Communication

Interthread communication involves synchronized threads to communicate with each other to pass information. In this mechanism, one of the threads is paused in its critical section to run and another thread is allowed to enter in the same critical section to be executed.

The inter-communication between threads is carried out by three methods, wait(), notify(), and notifyAll(), which can be called within the synchronized context.

Methods for Inter-Thread communication are as follows

- i. final void wait()
- ii. final void wait(long time milliseconds)
- iii. final void wait (long time milliseconds, int nanoseconds)
- iv. final void notify()
- v. final void notifyAll()

Example: ThreadDemo2.java

```
class BankAccount
{
    int accountNumber;
    static double balance;
    BankAccount(int n, double y)
    {
        accountNumber = n;
        balance = y;
    }
    synchronized void withDraw(int wd)
    {
        if(balance < wd)
            System.out.println("Less balance " + + balance + " is
            available; waiting to deposit more " );
        if(balance >= wd)
        {
            System.out.println("balance is available : "+ balance);
            balance = balance -wd;
            System.out.println("balance after withdrawal : " +
            balance);
        }
        try{
            wait();
        }catch(Exception e)
        {
            
```

```
}  
if(balance > wd)  
{  
    System.out.println("balance is available : "+ balance);  
    balance = balance - wd;  
    System.out.println("balance after withdrawal : " +  
balance);  
}  
}  
synchronized double deposit(int dp)  
{  
    System.out.println("Going to deposit: " + dp );  
    balance = balance + dp;  
    System.out.println("Balance after deposit = "+ balance);  
    notify();  
    return(balance);  
}  
}  
public class ThreadDemo2  
{  
    public static void main(String[] args)  
    {  
        final BankAccount ba = new BankAccount(2345, 1000.0);  
        new Thread()  
        {  
            public void run()  
            {
```



```
ba.withDraw(5000);  
}  
}.start();  
new Thread()  
{  
public void run()  
{  
ba.deposit(15000);  
}  
}.start();  
}  
}
```

```
C:\ >javac ThreadDemo2.java
```

```
C:\ >java ThreadDemo2
```

Less balance 1000.0 is available; waiting to deposit more

Going to deposit: 15000

Balance after deposit = 16000.0

balance is available : 16000.0

balance after withdrawal : 11000.0

1.9 Suspending Resuming and Stopping of Threads.

- In Java 1.1, the following three methods are defined

- suspend() - pause the operation of thread
- resume() - resume the operation of thread
- stop() - to terminate the thread

- These direct methods are very convenient to control the operation of threads in a multithread environment.

- However, in some situations, they may crash the program or cause serious damage to critical data.
- For example, if a thread has got the lock to a critical synchronized section of code and gets suspended, it will not release the lock for which other threads may be waiting.
- Instead of methods `suspend()` and `resume()`, the methods `wait()` and `notify()` are used.

```
class Counter
{
    int count=0;

    public synchronized void increment()
    {
        count++;
    }
}
```

Output: After adding synchronized keyword to increment() method

```
C:\>javac SyncDemo.java
```

```
C:\>java SyncDemo
```

```
Count = 10000
```

```
C:\>java SyncDemo
```

```
Count = 10000
```

```
C:\>java SyncDemo
```

```
Count = 10000
```

Module2:**Event Handling:****2.1 Events:**

- An event is an object that describes a state change in a source.
- Even driven is a consequence interaction of the user with the GUI. Some of the common interactions are moving the mouse, clicking the mouse, clicking a button, typing in a textfield etc.,

2.2 Event Sources:

- A source is an object that generates an event. This occurs when the internal state of that object changes in some way.
- Source may generate more than one type of event.

2.3 Event classes:

- Event classes are used to handle java event handling mechanism. At the root of the java event class hierarchy is **EventObject**, which is in java.util package. It is the superclass for all events. It provides a constructors as **EventObject(Object x)**

Methods provided by the class are

1. Object getSource(): returns the source of the event
2. String toString (): returns the string equivalent of the event.

- The class AWTEvent, defined with in the java.awt package, is subclass of EventObject.
- It is the suprclass of all AWT-based events used by the delegation model.
- Commonly used Event classes are
- ActionEvent, AdjustmentEvent, ComponentEvent, ContainerEvent, FocusEvent, InputEvent, ItemEvent, KeyEvent, MouseEvent, MouseWheelEvent, TextEvent, WindowEvent.

2.4 Event Listeners: A listener is an object that is notified when an event occurs. It has two requirements

- It must have been registered with one or more sources to receive notifications about the specific type of events.
- It must implement methods to receive and process the notifications. The methods that receive and process events are defined in a set of interfaces found in java.awt.event package.

Commonly used Event Listener Interfaces are

ActionListener,

AdjustmentListener
ComponentListener
ContainerListener
FocusListener
ItemListener
KeyListener
MouseListener
MouseMotionListener
TextListener
WindowFocusListener
WindowListener

2.5 Delegation event model:

- The Event Delegation Model defines a consistent mechanism to generate and process the events.
- The process is: **a source generates an event and sends it to one or more listeners.** In this scheme, the listener simply waits until it receives an event. Once an event is received, the listener processes the event and then returns.

2.6 Mouse Events:

MouseEvent Class: The MouseEvent class defines 8 types of mouse events. It defines the following integer constants that can be used to identify them:

1. `MOUSE_CLICKED` : The user clicked the mouse
2. `MOUSE_DRAGGED` : The user dragged the mouse
3. `MOUSE_ENTERED` : The mouse entered a component
4. `MOUSE_EXITED` : The mouse exited from a component
5. `MOUSE_MOVED` : The mouse moved
6. `MOUSE_PRESSED` : The mouse was pressed
7. `MOUSE_RELEASED` : The mouse was released
8. `MOUSE_WHEEL` : The mouse wheel was moved

Methods of MouseEvent Class:

1. int getX() : returns the X coordinates of the mouse when an event occurred
2. int getY(): returns the Y coordinates of the mouse when an event occurred.
3. Point getPoin(): returns the coordinates of the mouse.

MouseListener Interface:

This interface defines five methods:

1. void mouseClicked(MouseEvent me): it invokes if the mouse is pressed and released at the same point
2. void mouseEntered(MouseEvent me): it invokes when the mouse enters a component.
3. void mouseExited(MouseEvent me): it invokes when the mouse leaves the component.
4. void mousePressed(MouseEvent me): it invokes when the mouse is pressed.
5. void mouseReleased(MouseEvent me): it invokes when the mouse is released.

MouseMotionListener Interface:

This interface defines two methods:

1. void mouseDragged(MouseEvent me): it invokes when multiple times as the mouse is dragged.
2. void mouseMoved(MouseEvent me): it invokes when multiple times as the mouse is moved

Handling MouseEvents:

Example program to Display the position of x and y co-ordinates of the cursor movement using mouse.

Program:

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/* <applet code="Mouse" width=500 height=500> </applet> */
public class Mouse extends Applet
implements MouseListener,MouseMotionListener
{
    int X=0,Y=20;
    String msg="MouseEvents";
    public void init()
    {
```

```
        addMouseListener(this);
        addMouseMotionListener(this);
        setBackground(Color.black);
        setForeground(Color.red);
    }
    public void mouseEntered(MouseEvent m)
    {
        setBackground(Color.magenta);
        showStatus("Mouse Entered");
        repaint();
    }
    public void mouseExited(MouseEvent m)
    {
        setBackground(Color.black);
        showStatus("Mouse Exited");
        repaint();
    }
    public void mousePressed(MouseEvent m)
    {
        X=10;
        Y=20;
        msg="GEC";
        setBackground(Color.green);
        repaint();
    }
    public void mouseReleased(MouseEvent m)
    {
        X=10;
        Y=20;
        msg="Engineering";
        setBackground(Color.blue);
        repaint();
    }
```

```
}  
public void mouseMoved(MouseEvent m)  
{  
    X=m.getX();  
    Y=m.getY();  
    msg="College";  
    setBackground(Color.white);  
    showStatus("Mouse Moved");  
    repaint();  
}  
public void mouseDragged(MouseEvent m)  
{  
    msg="IT";  
    setBackground(Color.yellow);  
    showStatus("MouseMoved"+m.getX()+" "+m.getY());  
    repaint();  
}  
public void mouseClicked(MouseEvent m)  
{  
    msg="Students";  
    setBackground(Color.pink);  
    showStatus("Mouse Clicked");  
    repaint();  
}  
public void paint(Graphics g)  
{  
    g.drawString(msg,X,Y);  
}  
}
```

2.7 KeyEvents:

KeyEvent Class: A KeyEvent is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants:

- KEY_PRESSED
- KEY_RELEASED
- KEY_TYPED

The first two events are generated when any key is pressed or released. The last event occurs when a character is generated.

Methods of KeyEvent Class:

- **char getKeyChar()** : returns the character that was entered
- **int getKeyCode()** : returns the character code.

KeyListener Interface:

The interface defines three methods:

- **void keyPressed(KeyEvent ke):** it invokes when a key is pressed.
- **void keyReleased(KeyEvent ke):** it invokes when a key is released.
- **Void keyTyped(KeyEvent ke):** it invokes when a key is typed.

Handling KeyEvents:

Example program to handle keyboard events, which echoes keystrokes to the applet window and shows the status of each key event in the status bar.

Program:

```
/* <Applet code="Key.class" width=4000 height=4000> </Applet> */
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
public class Key extends Applet implements KeyListener
{
    String msg="";
    public void init()
    {
        setBackground(Color.green);
    }
}
```



```
        addKeyListener(this);
    }
    public void keyPressed(KeyEvent k)
    {
        showStatus("Key pressed");
        if(k.getKeyCode()==KeyEvent.VK_ENTER)
            showStatus("Enter key is pressed:");
        repaint();
    }
    public void keyReleased(KeyEvent k)
    {
        showStatus("Key Up or Key Released");
    }
    public void keyTyped(KeyEvent k)
    {
        msg+=k.getKeyChar();
        repaint();
    }
    public void paint(Graphics g)
    {
        g.setFont(new Font("Arial",Font.BOLD,30));
        g.drawString("Key Typed is:"+msg,20,300);
    }
}
```

Assignment cum Tutorial questions**Part-A****Objective Questions (1M)****Module1: Multithreaded Programming**

1. Which of this method can be used to make the main thread to be executed last among all the threads? []
a) stop() b) sleep()
c) join() d) call()
2. Which of this method is used to find out that a thread is still running or not? []
a) run() b) Alive()
c) isAlive() d) checkRun()
3. What is the default value of priority variable MIN_PRIORITY AND MAX_PRIORITY? []
a) 0 & 256 b) 0 & 1
c) 1 & 10 d) 1 & 256
4. What is synchronization in reference to a thread? []
a) It's a process of handling situations when two or more threads need access to a shared resource
b) It's a process by which many thread are able to access same shared resource simultaneously
c) It's a process by which a method is able to access many different threads simultaneously
d) It's a method that allow too many threads to access any information require
5. Which of these method is used to explicitly set the priority of a thread? []
a) set() b) make()
c) setPriority() d) makePriority()
6. Which of these method is used to implement Runnable interface? []
a) stop() b) run()
c) runThread() d) stopThread()
7. Which of these method is used to begin the execution of a thread? []
a) run() b) start()
c) runThread() d) startThread()
8. Which of these method of Thread class is used to find out the priority given to a thread? []
a) get() b) ThreadPriority()
c) getPriority() d) getThreadPriority()
9. What is multithreaded programming? []
a) It's a process in which two different processes run simultaneously
b) It's a process in which two or more parts of same process run simultaneously
c) It's a process in which many different process are able to access same information
d) It's a process in which a single process can access information from many sources
10. Which of these are types of multitasking? []
a) Process based b) Thread based

- c) Process and Thread based d) None of the mentioned
11. Thread priority in Java is? []
a) Integer b) Float
c) double d) long
12. What will happen if two thread of the same priority are called to be processed simultaneously? []
a) Anyone will be executed first lexographically
b) Both of them will be executed simultaneously
c) None of them will be executed
d) It is dependent on the operating system
13. Which of these statements is incorrect? []
a) By multithreading CPU idle time is minimized, and we can take maximum use of it
b) By multitasking CPU idle time is minimized, and we can take maximum use of it
c) Two thread in Java can have the same priority
d) A thread can exist only in two states, running and blocked
14. What requires less resources? []
a) Thread b) Process
c) Thread and Process d) Neither Thread nor Process
15. What decides thread priority? []
a) Process b) Process scheduler
c) Thread d) Thread scheduler
16. What does not prevent JVM from terminating? []
a) Process b) Daemon Thread
c) User Thread d) JVM Thread
17. What is true about time slicing? []
a) Time slicing is OS service that allocates CPU time to available runnable thread
b) Time slicing is the process to divide the available CPU time to available runnable thread
c) Time slicing depends on its implementation in OS
d) Time slicing allocates more resources to thread
18. What is true about threading? []
a) run() method calls start() method and runs the code
b) run() method creates new thread
c) run() method can be called directly without start() method being called
d) start() method creates new thread and calls code written in run() method
19. Which of the following is a correct constructor for thread? []
a) Thread(Runnable a, String str)
b) Thread(int priority)
c) Thread(Runnable a, int priority)
d) Thread(Runnable a, ThreadGroup t)
20. Which of the following will ensure the thread will be in running state? []
a) yield() b) notify()

- c) wait() d) Thread.killThread()
21. Which of these keywords are used to implement synchronization? []
- a) synchronize b) syn
c) synch d) synchronized
22. Which of this method is used to avoid polling in Java? []
- a) wait() b) notify()
c) notifyAll() d) all of the mentioned
23. Which of these method wakes up the first thread that called wait()? []
- a) wake() b) notify()
c) start() d) notifyAll()
24. Which of these method wakes up all the threads? []
- a) wakeAll() b) notify()
c) start() d) notifyAll()

Module2: Event Handling

1. AWT stands for []
- (a) Applet Windowing Toolkit
(b) Abstract Windowing Toolkit
(c) Absolute Windowing Toolkit
(d) None of the above
2. Which object can be constructed to show any number of choices in the visible window? []
- (a) Labels (b) Choice (c) List (d) Checkbox
3. Which class provides many methods for graphics programming? []
- (a) java.awt (b) java.Graphics
(c) java.awt.Graphics (d) None of the above
4. _____ Layout arranges the components as a deck of cards such that only one component is visible at a time []
- (a) CardLayout (b) BorderLayout (c) FlowLayout (d) GridLayout
5. At the top of the AWT hierarchy is the _____ class. []
- (a) Window (b) Component (c) Panel (d) Frame
6. AWT classes are contained in the _____ package []
- (a) java.awt (b) java.Awt

(c) java.classes.awt (d) java.pacakge.awt

7. BorderLayout class has __regions to add components to it []

(a) 4 (b) 7 (c) 5 (d) 8

8. By default FlowLayout uses _____justification. []

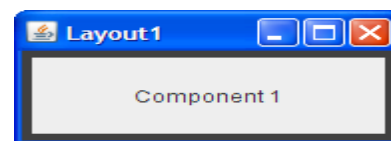
(a) Left (b) Right (c) Center (d) Top

9. By default page-up and page-down increment of scrollbar is_ _[]

(a) 5 (b) 10 (c) 7 (d) 6

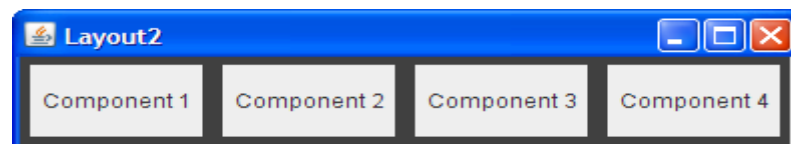
10. In each of the following questions, choose the layout manager(s) most naturally suited for the described layout. The container has one component that should take up as much space as possible

[]



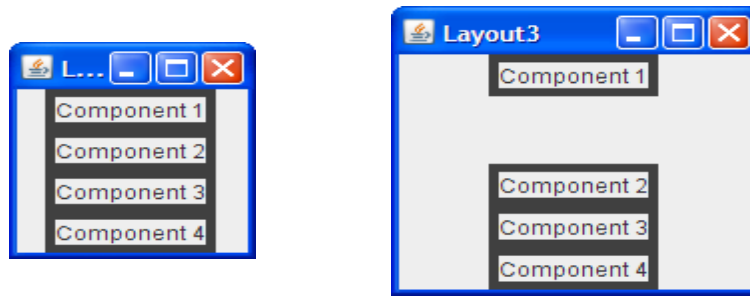
a). BorderLayout b). GridLayout c). GridBagLayout d). a and b e). b and c

10. The container has a row of components that should all be displayed at the same size, filling the container's entire area. []



a). FlowLayout b). GridLayout c). BoxLayout d). a and b

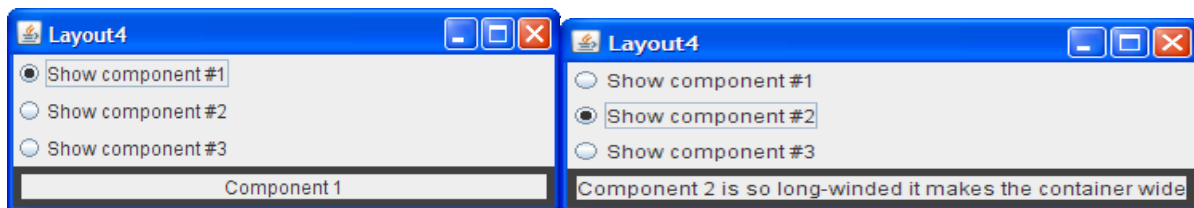
11. The container displays a number of components in a column, with any extra space going between the first two components. []



- a). FlowLayout b). BorderLayout c). GridLayout d). BorderLayout

12. The container can display three completely different components at different times, depending perhaps on user input or program state. Even if the components' sizes differ, switching from one component to the next shouldn't change the amount of space devoted to the component.

[]



- a). SpringLayout b). BorderLayout c). CardLayout d). GridBagLayout

Part-B

Short Answer Questions (2M)

Module1: MultiThreading

1. What is the thread? [BL2]
2. Differentiate between process and thread? [BL3]
3. What is multithreading? [BL2]
4. What do you understand by inter-thread communication? [BL2]
5. What is the purpose of wait() method in Java? [BL2]
6. Why must wait() method be called from the synchronized block? [BL3]
7. What are the advantages of multithreading? [BL2]
8. What does join() method? [BL2]
9. Describe the purpose and working of sleep() method. [BL2]
10. What is the difference between wait() and sleep() method? [BL3]
11. Is it possible to start a thread twice? [BL3]
12. Can we call the run() method instead of start()? [BL3]
13. What about the daemon threads? [BL2]
14. What is the synchronization? [BL2]
15. What is the difference between notify() and notifyAll()? [BL3]

Module2: Event Handling.

1. Differentiate between Event and event source. [BL2]
2. List any four mouse events. [BL2]
3. Differentiate between application program and applet programming. [BL3]

Part-C**Descriptive Questions (5M)****Module 1: Multithreaded Programming**

1. What are the states in the lifecycle of a Thread? [BL2]
2. Write a java program to create multiple threads. What are the advantages of multithreading? [BL3]
3. Differentiate between the Thread class and Runnable interface for creating a Thread? [BL3]
4. What about the daemon threads? Can we make the user thread as daemon thread if the thread is started? [BL2]
5. Explain thread synchronization with respect to multithreading. Why is it important? [BL3]
6. “Intercommunication between thread is relatively economical than processes” justify this statement. [BL2]
7. What is thread scheduling? How to perform this by setting priorities to threads. explain with an example program [BL3]

Module 2: Event Handling

1. Explain delegation event model. [BL3]
2. Discuss different types of event listeners. [BL3]
3. Compare between event and event sources. [BL3]
4. Write a java program to implement mouse events. [BL3]
5. Differentiate between mouse and mouse motion event. [BL3]
6. Write a java program to implement key events. [BL3]